

Threshold Receipt-Free Voting with Server-Side Vote Validation

Thi Van Thao Doan, Olivier Pereira, and Thomas Peters

UCLouvain – ICTEAM – Crypto Group
B-1348 Louvain-la-Neuve – Belgium
{thi.doan, olivier.pereira, thomas.peters}@uclouvain.be

Abstract. Proving the validity of ballots is a central element of verifiable elections. Such proofs can however create challenges when one desires to make a protocol receipt-free.

We explore the challenges raised by validity proofs in the context of protocols where threshold receipt-freeness is obtained by secret sharing an encryption of a vote between multiple authorities. In such contexts, previous solutions verified the validity of votes by decrypting them after passing them through a mix-net. This approach however creates subtle privacy risks, especially when invalid votes leak structural patterns that threaten receipt-freeness.

We propose a different approach of threshold receipt-free voting in which authorities re-randomize ballot shares then jointly compute a ZK proof of ballot validity before letting the ballots enter a (possibly homomorphic) tallying phase. Our approach keeps the voter computational costs limited while offering verifiability and improving the ballot privacy of previous solutions.

We present two protocols that enable a group of servers to verify and publicly prove that encrypted votes satisfy some validity properties: **MiniMix**, which preserves prior voter-side behavior with minimal overhead, and **HomoRand**, which requires voters to submit auxiliary data to facilitate validation over large vote domains. We show how to use our two protocols within a threshold receipt-free voting framework. We provide formal security proofs and efficiency analyses to illustrate trade-offs in our designs.

Table of Contents

Threshold Receipt-Free Voting with Server-Side Vote Validation	1
<i>Thi Van Thao Doan, Olivier Pereira, and Thomas Peters</i>	
1 Introduction	2
2 Building Blocks	4
3 Verifiable Ciphertext Validity in MPC	5
3.1 MPC-MiniMix Protocol	5
3.2 MPC-HomoRand Protocol	6
4 Threshold Receipt-Free Voting	9
5 Voting Schemes	11
5.1 Overview	11
5.2 The MiniMix Construction	12
5.3 The HomoRand Construction	13
5.4 Efficiency Discussion	14
6 Security of the Voting Schemes	16
7 Conclusion	17
A Details of the HomoRand Construction	18
A.1 Computational Setting	18
A.2 Groth-Sahai Proofs	19
A.3 The HomoRand Construction	19
B MPC protocols' Correctness	22
B.1 MPC-MiniMix	22
B.2 MPC-HomoRand	23
C Security of the Voting Schemes	25
C.1 Threshold Correctness	25
C.2 Threshold Receipt-Freeness	27

1 Introduction

Receipt-freeness (RF), is a central property of voting systems that ensures that voters cannot prove how they voted to a third party. Since its formal introduction by Benaloh and Tuinstra [2], RF has been the subject of extensive research, especially because of the tension that it creates with the transparency and verifiability of the election process.

A standard approach first explored by Hirt [13] consists in asking voters to encrypt their vote and submit the ciphertext to a ballot processing server that privately re-randomizes the ciphertext before posting it on a public bulletin-board. Since the re-randomization removes the voter's knowledge of the encryption randomness, voters cannot offer any evidence of the ciphertext content to any third party. In Hirt's solution, the ballot processing server must also send a designated verifier proof that it re-randomized the ballot without modifying its content. Subsequent works have aimed to eliminate the need for sending

such a proof while preserving RF using various types of randomizable signatures [3, 4, 7–9]. However, in all these works, receipt-freeness depends on a single trusted ballot processing server: if the ballot processing server leaks the randomness used to re-randomize the ballot, RF is lost (though privacy and verifiability would still be preserved).

In order to remove this single point of failure, Doan et al. [10] introduced a threshold receipt-free voting framework, leveraging multiple ballot processing servers (randomizers) to decentralize trust. Their system ensures RF and correctness as long as the number of corrupted randomizers remains below a threshold. Moreover, it maintains universal verifiability via mixnets.

However, Doan’s protocol does not include any ballot validity proof in the ballot submission process: the validity of the votes is only verified when the mixed ciphertexts are decrypted. As a result, a coercer or a vote buyer could ask to see a very unusual invalid vote among the decrypted ballots (it could simply ask to encrypt a large randomly chosen value). This strategy, akin to the Italian attack [2], provides a limited form of receipt, in the sense that the voter can only demonstrate that he submitted an invalid vote, but it would still be desirable to avoid it whenever possible.

Worst situations may happen when the set of valid votes is very large. For instance, if a vote is by approval among 100 candidates, a voter could simply demonstrate how he voted by submitting a highly unusual approval pattern, which will come uniquely in the tally. This limitation highlights the need for alternative solutions that preserve the security requirements of threshold receipt-freeness while avoiding direct decryption of all individual votes, whether they are valid or not.

Tallying processes based on the homomorphic aggregation of ballots instead of a mixnet can offer a solution to these problems in the context of approval voting (and in other cases), while tally-hiding protocols offer more general solutions: in these protocols, individual votes are never decrypted.

Contributions. We propose a new paradigm for threshold receipt-free voting that addresses a core limitation in [10]: the lack of vote validity enforcement before tallying. Instead of requiring voters to generate complex zero-knowledge proofs at ballot submission time, we defer validity checking to a *pre-tally phase*, executed after combined ciphertexts are reconstructed but before they are tallied.

In particular, each voter secret-shares their vote and encrypts the shares under a traceable RF encryption scheme (TREnc) [7], then submits the resulting ciphertexts (ballot shares) to a set of processing servers. These servers rerandomize the ballot shares and post them to a public bulletin board. After voting concludes, any party (e.g., the talliers) reconstructs each final ciphertext by combining a threshold of rerandomized ballot shares. These ciphertexts are then validated in a *pre-tally phase* via a multi-party computation (MPC) protocol that checks whether each encrypts a vote in the prescribed domain, without revealing the vote or auxiliary information. Valid ballots are then forwarded to a standard tallying procedure. This paradigm shift decouples vote-domain enforcement from the voter’s submission step and moves it to a joint server-side

protocol, overcoming limitations of threshold RF settings, where secret sharing and rerandomization rendered traditional ZK-based validity checks infeasible. We realize this paradigm through two constructions:

- **MiniMix**: mirrors the architecture of Doan et al. [10], with voters submitting encrypted shares of their vote under **TREnc**, and performs MPC-based pre-tally validation over the reconstructed ciphertext.
- **HomoRand**: allows voters to submit auxiliary information that facilitates more efficient validity checks in certain settings, while preserving RF.

These constructions extend the threshold RF model of [10] beyond mixnet-based designs to support homomorphic tallying over binary and general vote domains. **HomoRand** achieves asymptotically superior pre-tally performance (logarithmic in the bit-length required to represent the domain span), but with greater voter-side computation; **MiniMix**, by contrast, is optimized for low client overhead, making it well-suited to moderate domain sizes or when client efficiency is critical.

Unlike general-purpose frameworks for collaborative zero-knowledge, such as that of Ozdemir and Boneh [15], which adapt ZK-SNARKs to distributed-witness settings, our protocols are tailored to voting-specific constraints. Here, the witness originates from a potentially malicious voter, and correctness and privacy must be enforced jointly, without compromising RF. Notably, the RF definition we target does not cover scenarios where an adversary prevents voting, such as forcing abstention or invalid ballots.

Remark. Although our protocols are motivated by [10], our pre-tally validation technique is broadly applicable. For instance, one could remove the 0/1 proofs from the CGS97 protocol [6] and insert a round of **MiniMix** to enforce ballot validity. Such a substitution would functionally ensure well-formed ballots, shifting the computational efforts from voting clients to servers.

2 Building Blocks

Secret Sharing. We use a standard t -out-of- n threshold secret sharing scheme [16], consisting of two algorithms. The sharing algorithm **Share**(n, t, m) splits a secret m into n shares $(m_1, \dots, m_n)$ such that any subset of at least t shares can reconstruct m , while any set of fewer than t shares reveals no information about m . The reconstruction algorithm **Combine**($n, t, m_1, \dots, m_t$) uses Lagrange interpolation to recover m from any t shares.

Traceable Receipt-Free Encryption (TREnc). **TREnc** [7] is a public key encryption scheme (**Gen**, **Enc**, **Dec**), augmented with a 5-tuple of algorithms: **LGen**, on input a security parameter λ and a public encryption key $\mathbf{pk}$, outputs a link key $\mathbf{lk}$; **LEnc** encrypts a message m using $(\mathbf{pk}, \mathbf{lk})$ and outputs a ciphertext **CT**. **Trace** outputs the trace τ of **CT**. **Rand** randomizes **CT** to output another ciphertext. **Ver** checks if a ciphertext is valid and outputs 1 if true, and 0 otherwise.

Informally, a **TREnc** scheme is *traceable* if no efficient adversary can produce a second ciphertext with the same trace as a given ciphertext, yet which decrypts

to a different message. It is *TCCA-secure* (Traceable Chosen-Ciphertext Attack secure) if re-randomized ciphertexts are indistinguishable from fresh ciphertexts with the same trace, even in the presence of a restricted decryption oracle. We assume it is straightforward to extract specific parts of the TREnc’s ciphertext CT. In particular:

- Strip(pk, CT):** Returns the CPA-encryption component of CT. For instance, in TREnc [7], this component is $\mathbf{c} = (c_0, c_1, c_2) = (G^m f^\theta, g^\theta, h^\theta)$, where m is the message, $\theta \leftarrow \mathbb{Z}_p$, and $(G, f, g, h) \in \text{pk}$.
- CPA(pk, m ; r):** Computes the CPA component of a TREnc ciphertext on message m with randomness r under TREnc’s public key pk. We note that this algorithm can be used independently of TREnc.

We further assume that the underlying CPA encryption is additively homomorphic over the message space $\mathbb{Z}_p$ and that the encryption operates over a cyclic group of prime order p . That is, given $\mathbf{C}_1 = \text{CPA}(\text{pk}, m_1; r_1)$ and $\mathbf{C}_2 = \text{CPA}(\text{pk}, m_2; r_2)$, one can compute $\mathbf{C}_1 \cdot \mathbf{C}_2 = \text{CPA}(\text{pk}, m_1 + m_2; r_1 + r_2)$ (up to group notation). In many cases, we omit the randomness when it is sampled uniformly at random. In the rest of the paper, we use the prefix TREnc. to indicate that an algorithm belongs to the TREnc suite.

Non-Interactive Proofs. Informally, a proof for an NP relation R is a protocol by which a prover P convinces a probabilistic polynomial-time (PPT) verifier V that $\exists w : R(w, x) = 1$, where x is called a statement, and w is a witness for x . In the non-interactive setting, the proof consists of a single message from P to V and proceeds as follows. A setup algorithm $\text{PrfSetup}(1^\lambda)$, on input a security parameter λ , generates a common reference string (CRS) σ . The prover then computes $\text{PrfProve}(\sigma, x; w)$, outputting a proof π if $R(x, w) = 1$, or $\perp$ otherwise. Finally, the verifier checks the proof via $\text{PrfVerify}(\sigma, x, \pi)$ and outputs 1 if the proof is valid, and 0 otherwise.

A non-interactive zero-knowledge proof of knowledge (NIZKPoK) satisfies completeness, knowledge soundness, and zero-knowledge [11].

3 Verifiable Ciphertext Validity in MPC

We present two MPC-based constructions for verifying whether a ciphertext $\mathbf{C} = \text{CPA}(\text{PK}, m)$, under a CPA-secure encryption scheme with the corresponding decryption $\text{Dec}(\text{SK}, \mathbf{C}) = m$, encrypts a message m satisfying a given constraint (e.g., $m \in \{v_1, \dots, v_d\} \subset \mathbb{Z}_p$). The protocol reveals only the outcome of the check and leaks no additional information about the underlying plaintext or any related value, even in the case of failure. In the following, we describe two such approaches in detail.

3.1 MPC-MiniMix Protocol

The first construction, referred to as MPC-MiniMix (Algorithm 1), employs an OR-proof mechanism to verify message validity. The core idea is to check whether

any of the ciphertexts $C_j = \text{CPA}(\text{PK}, m - v_j)$ encrypts the value 0, where each C_j is derived as $C \cdot \text{CPA}(\text{PK}, v_j)^{-1}$ for $j \in [d]$, exploiting the additive homomorphism of the CPA scheme. Conceptually, this construction generalizes plaintext equivalence proofs [14] to enable a form of privacy-preserving range verification.

The sender first encrypts the message m , from which all parties can compute the ciphertexts $\{C_j\}_{j \in [d]}$. The parties then engage in a collaborative verification process, structured as a simplified mixnet operating over the d ciphertexts. Each party $\mathcal{T}_k$, *in turn*, shuffles the ciphertexts (lines 4–9), with the output of $\mathcal{T}_k$ serving as the input to $\mathcal{T}_{k+1}$, for all $k \in [T - 1]$, where T denotes the number of parties. Each shuffle includes a re-randomization step (line 7), ensuring that—under honest execution—the ciphertext encrypting 0 becomes unlinkable to its initial position, thereby preserving zero-knowledge.

A significant challenge arises when the message m is a carefully crafted value. In such cases, decryption may reveal not only the success or failure of the range verification but also partial information about m , specifically $m - v_j$ for some j . This could potentially assist the adversary in extracting structural patterns and re-identify the voter. To mitigate this risk, we incorporate a masking step following the shuffle: after shuffling, each party $\mathcal{T}_k$ raises each ciphertext to a fresh random exponent $\alpha_j^{(k)}$ and proves correctness via a ZKPoK $\pi^{(k)}$ (lines 10–14). These ciphertexts and proofs are then broadcast (line 15). The shuffle proof can be instantiated using any standard zero-knowledge proof system for shuffle correctness (e.g., [1, 17]), while the re-randomization proof can be implemented via a standard Σ -protocol.

We assume an honest majority setting. If the majority finds that a proof fails verification (line 16), the corresponding party $\mathcal{T}_k$ is excluded from the protocol, and the output of the last honest party is used to proceed. Although the shuffling and exponentiation phases are presented separately for clarity and to support later comparative analysis (Section 5.4), they can be efficiently merged into a single phase with a unified zero-knowledge proof. Finally, the parties jointly decrypt the resulting ciphertexts (line 20). If the message m is valid, at least one ciphertext will decrypt to 0 while being different of $\text{CPA}(\text{PK}, 0; 0)$; otherwise all decrypted values appear as random group elements, preventing any information leakage about m .

3.2 MPC-HomoRand Protocol

In this approach, we leverage pairings to ensure that the encrypted message m lies within a prescribed integer interval $[v_1, v_d] \subset \mathbb{Z}_p$. Without loss of generality, let l be the smallest number of bits such that the span of the domain satisfies $v_d - v_1 < 2^l$. To prove that $m \in [v_1, v_d]$ (inclusive), it suffices to verify that $m - v_1 \in [0, 2^l - 1]$ and $v_d - m \in [0, 2^l - 1]$. For simplicity, here we demonstrate how to prove that $m \in [0, 2^l - 1]$. This is accomplished by having the sender decompose m into an l -bit string $b_1 b_2 \dots b_l$, and separately encrypt each bit in two distinct source groups $\mathbb{G}$ and $\hat{\mathbb{G}}$. Subsequently, a MPC protocol operates in the target group $\mathbb{G}_T$ to jointly verify that each encrypted bit b_j satisfies $b_j \in \{0, 1\}$ for all $j \in [l]$. This binary encoding allows the sender’s computational effort to scale logarithmically with the bit length l , i.e., $O(\log l)$.

Algorithm 1 MPC-MiniMix: Sequential Multi-Party Randomization & Verification

```

1: procedure MPC-MiniMix(PK, SK,  $C = \{C_j\}_{j \in [d]}$ )
2:   Initialize  $C^{(0)} \leftarrow C$  ▷ i.e.,  $C_j^{(0)} \leftarrow C_j$  for  $j \in [d]$ 
3:   for  $k = 1$  to  $T$  do ▷ Each party acts sequentially
4:     Sample at random a permutation  $P_k$  of  $\{1, \dots, d\}$ 
5:     for  $j = 1$  to  $d$  do ▷ Shuffle the ciphertexts
6:        $s_j^{(k)} \leftarrow \mathbb{Z}_p \setminus \{0\}$ 
7:        $C_{P_k(j)}^{(k)} \leftarrow C_j^{(k-1)} \cdot \text{CPA}(\text{PK}, 1_{\mathbb{G}}; s_j^{(k)})$ 
8:     end for
9:      $\pi_{sf}^{(k)} \leftarrow \text{PrfProve}(\text{PK}, \{C_j^{(k)}, C_j^{(k-1)}\}_{j \in [d]}; P_k, \{s_j^{(k)}\}_{j \in [d]})$ 
10:    for  $j = 1$  to  $d$  do ▷ Apply random factors
11:      Sample random values  $\alpha_j^{(k)}, r_j^{(k)} \leftarrow \mathbb{Z}_p \setminus \{0\}$ 
12:       $C_j^{(k)} \leftarrow (C_j^{(k-1)})^{\alpha_j^{(k)}} \cdot \text{CPA}(\text{PK}, 1_{\mathbb{G}}; r_j^{(k)})$ 
13:       $\pi_{rd,j}^{(k)} \leftarrow \text{PrfProve}(\text{PK}, C_j^{(k)}, C_j^{(k-1)}; \alpha_j^{(k)}, r_j^{(k)})$ 
14:    end for
15:    Publish  $C^{(k)} = \{C_j^{(k)}\}_{j \in [d]}$  and  $\pi^{(k)} = (\pi_{sf}^{(k)}, \{\pi_{rd,j}^{(k)}\}_{j \in [d]})$ 
16:    if  $\text{PrfVer}(\pi^{(k)}) = 0$  then return  $\perp$ 
17:    end if
18:  end for
19:  for  $j = 1$  to  $d$  do
20:     $m_j \leftarrow \text{Dec}(\text{SK}, C_j^{(T)})$  ▷ Decrypt the final ciphertexts
21:    if  $m_j = 0$  then return 1 ▷ Return 1 if any decrypted value is 0
22:    end if
23:  end for
24:  return 0 ▷ Return 0 if no ciphertext decrypts to 0
25: end procedure

```

More precisely, the sender encodes each bit $b_j \in \{0, 1\}$ by encrypting G^{b_j} and $\hat{G}^{b_j}$ under public keys PK_1 and PK_2 , respectively, yielding ciphertexts $c_j = \text{CPA}(\text{PK}_1, G^{b_j}) \in \mathbb{G}$ and $\hat{c}_j = \text{CPA}(\text{PK}_2, \hat{G}^{b_j}) \in \hat{\mathbb{G}}$. The parties then jointly evaluate the expression $e(G, \hat{G})^{\sum_{j \in [l]} \gamma_j b_j (1 - b_j)} \stackrel{?}{=} 1_{\mathbb{G}_T}$, where $e : \mathbb{G} \times \hat{\mathbb{G}} \rightarrow \mathbb{G}_T$ denotes a bilinear pairing, and each $\gamma_j \leftarrow \mathbb{Z}_p$ is a blinding factor. This check succeeds if and only if all bits are well-formed, i.e., $b_j \in \{0, 1\}$ for every $j \in [l]$. Crucially, the use of masking randomness ensures that the check does not reveal the index or value of any invalid bit, thereby preserving privacy even in the case of malformed inputs. In the MPC setting, the blinding factors γ_j are additively shared across parties: each party $\mathcal{T}_k$ for $k \in [T]$ locally samples $\gamma_{k,j} \leftarrow \mathbb{Z}_p$, and $\gamma_j = \sum_{k \in [T]} \gamma_{k,j}$. These values are generated *in parallel* across all parties.

We instantiate this technique in the MPC-HomoRand protocol, specified in Algorithms 2 and 3, using a concrete CPA encryption scheme. Let the $\text{PK} = (G, f, g, h) \in \mathbb{G}^4$, with secret key $\text{SK} = (\alpha, \beta) \in \mathbb{Z}_p^2$ and $f = g^\alpha h^\beta$. Encryption of a message $m \in \mathbb{Z}_p$ with randomness r proceeds as $c = \text{CPA}(\text{PK}, m; r) = (c_0, c_1, c_2) = (G^m f^r, g^r, h^r)$, and decryption yields $\text{Dec}(\text{SK}, c) = c_0 \cdot c_1^{-\alpha} \cdot c_2^{-\beta} = G^m$.

Algorithm 2 MPC-HomoRand.Part1: Parallel Randomization by each $\mathcal{T}_k$ ($k \in [T]$)

```

1: procedure MPC-HomoRand.Part1( $\text{PK}_1, \text{PK}_2, C$ )      ▷ Input:  $C = \{(c_j, \hat{c}_j)\}_{j \in [l]}$ 
2:   for  $j = 1$  to  $l$  do                                ▷ For each vote bit
3:      $\gamma_{k,j}, \lambda_{k,j}, r_{k,j}, s_{k,j} \leftarrow \mathbb{Z}_p \setminus \{0\}$ 
4:      $c''_{k,j} \leftarrow c_j^{\gamma_{k,j}} \cdot \text{CPA}(\text{PK}_1, G^{\lambda_{k,j}}; r_{k,j})$       ▷ Apply random factors
5:      $\hat{c}''_{k,j} \leftarrow (\text{CPA}(\text{PK}_2, \hat{G}; 0)/\hat{c}_j)^{\lambda_{k,j}} \cdot \text{CPA}(\text{PK}_2, 1_{\hat{G}}; s_{k,j})$  ▷ Apply random factors
6:      $\pi_{k,j} \leftarrow \text{PrfProve}(\text{PK}_1, \text{PK}_2, C, c''_{k,j}, \hat{c}''_{k,j}; \gamma_{k,j}, \lambda_{k,j}, r_{k,j}, s_{k,j})$ 
7:      $C''_{k,j} \leftarrow (c''_{k,j}, \hat{c}''_{k,j}, \pi_{k,j})$ 
8:   end for
9:   return  $C''_k = \{C''_{k,j}\}_{j \in [l]}$ 
10: end procedure

```

Algorithm 3 MPC-HomoRand.Part2: Multi-Party Verification

```

1: procedure MPC-HomoRand.Part2( $\text{PK}_1, \text{PK}_2, \text{SK}_1, \text{SK}_2, C, \{C''_k\}_{k \in [T]}$ )
2:   for  $j = 1$  to  $l$  do                                ▷ For each vote bit
3:     for  $k = 1$  to  $T$  do
4:       if  $\text{PrfVer}(\text{PK}, C''_{k,j}, \pi_{k,j}) = 0$  then return  $\perp$ 
5:     end if
6:   end for
7:    $c''_j = (c''_{0j}, c''_{1j}, c''_{2j}) \leftarrow \prod_{k=1}^T c''_{k,j}$       ▷ Aggregate the ciphertexts
8:    $\hat{c}''_j = (\hat{c}''_{0j}, \hat{c}''_{1j}, \hat{c}''_{2j}) \leftarrow \prod_{k=1}^T \hat{c}''_{k,j}$ 
9:    $X_j \leftarrow \text{Dec}(\text{SK}_1, c''_j)$ 
10:   $\vec{c}'_j = (\vec{c}'_{0j}, \vec{c}'_{1j}, \vec{c}'_{2j}) \leftarrow \text{CPA}(\text{PK}_2, \hat{G}; 0)/\hat{c}_j$ 
11:   $x_j, y_j, z_j \leftarrow e(X_j, \vec{c}'_{0j})/e(G, \hat{c}''_{0j}), e(X_j, \vec{c}'_{1j})/e(G, \hat{c}''_{1j}), e(X_j, \vec{c}'_{2j})/e(G, \hat{c}''_{2j})$ 
12: end for
13:   $(x, y, z) \leftarrow (\prod_{j=1}^l x_j, \prod_{j=1}^l y_j, \prod_{j=1}^l z_j)$ 
14:   $Y \leftarrow \text{Dec}(\text{SK}_2, (x, y, z))$       ▷ Decrypt the aggregated result
15:  return  $Y = 1_{\mathbb{G}_T} ? 1 : 0$ 
16: end procedure

```

In Algorithm 2, each party independently masks the ciphertexts in parallel and proves correctness via a ZKPoK $\{\pi_{k,j}\}_{j \in [l]}$, which are broadcast alongside the resulting ciphertexts. Algorithm 3 specifies the subsequent verification phase: each proof is validated (line 4), and any party that fails verification is excluded. The combination of valid ciphertexts (lines 7–8) is computed solely from the outputs of honest parties, and the final values are jointly decrypted (line 14). The values (x_j, y_j, z_j) computed in line 11 are used to securely mask the term $b_j(1 - b_j)$ which would otherwise leak information if $b_j \notin \{0, 1\}$.

Although the description employs a specific CPA encryption scheme for concreteness (see Appendix A.3 for a proof of correctness), our construction generalizes to any additive homomorphic CPA scheme. The core idea, securely verifying bit decomposition without leakage, remains applicable in broader cryptographic contexts.

4 Threshold Receipt-Free Voting

We adopt the voting system model of Doan et al. [10], which supports multiple ballot processing servers and ensures both threshold receipt-freeness and threshold correctness. We then review the first threshold receipt-free scheme from [10], whose limitations motivate our new constructions in Section 5.

Definition 4.1 (Voting System [10]). *A voting system with n ballot processing servers consists of the PPT algorithms: (SetupElection, Vote, ProcessBallot, TraceBallot, Valid, Append, Publish, VerifyVote, Tally, VerifyResult), and a result function $\rho_m : \mathbb{V}^m \cup \{\perp\} \rightarrow \mathbb{R}$, where $\mathbb{V}$ is the vote domain and $\mathbb{R}$ is the result space.*

The election is initialized via **SetupElection**. Each voter runs **Vote** to generate a ballot consisting of n shares, one per ballot processing server. Each share is independently randomized by a distinct server using **ProcessBallot**. A tracking code is derived via **TraceBallot**, enabling voters to later trace their ballots on the public bulletin board PBB. Validated shares are collected and appended to the ballot box BB using **Append**, then made publicly available on PBB via **Publish**. Voters verify correct recording using **VerifyVote** and their tracking codes. The tally is computed over valid ballots, checked by **Valid**, using **Tally**, and its correctness is publicly verifiable via **VerifyResult**.

Threshold Receipt-Freeness (t_{rf}). The definition of threshold RF proceeds in two parts. The first ensures that no adversary—despite corrupting up to t_{rf} out of n ballot processing servers—can coerce a voter into producing a receipt that convincingly proves how they voted. This guarantee holds even if the adversary learns the voter’s randomness or attempts to bias the ballot construction. The second part, extends the indistinguishability-based receipt-freeness notion of Devillez et al. [7] to the threshold setting. It formalizes the requirement that even if a voter colludes with up to t_{rf} servers and deviates arbitrarily from the honest ballot distribution, a third party, given full knowledge of how the ballot was constructed and access to the public bulletin board, should not be able to determine whether the voter submitted that ballot or a different one encoding another vote.

Threshold Correctness (t_{corr}). Threshold correctness ensures that the choices of honest voters are faithfully reflected in the final tally, even in the presence of limited adversarial control over the ballot processing servers. Concretely, it guarantees that as long as at least $n - t_{corr}$ processed shares of each honestly generated ballot appear on the public bulletin board—regardless of whether they were handled by malicious servers—the election outcome remains uniquely determined. That is, the adversary should not be able to construct two sets of valid-looking ballot boxes that lead to different outcomes.

The Doan et al.’s Voting Scheme. The first protocol to simultaneously achieve threshold receipt-freeness and correctness was proposed by Doan et al. [10] (E-Vote-ID 2024). Their construction, illustrated in Figure 1, introduces a threshold receipt-free voting system that significantly reduces trust assumptions on ballot randomizers.

In their scheme, each voter secret-shares their vote, encrypts each share using a TREnc (Section 2), and submits the resulting ballot shares $\{b_i\}_{i \in [n]}$ to a set of independent randomizers. Each randomizer then rerandomizes one ballot share and output b'_i made available on the public board BB. After the voting phase,

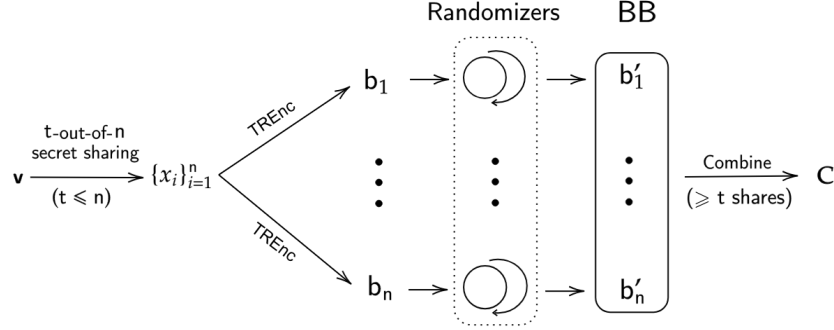


Fig. 1: Doan et al.'s voting scheme.

the talliers (or any observer) extract standard CPA components from the TREnc outputs, and use Lagrange interpolation to reconstruct an encryption of the original vote. This process results in a final ciphertext C that is then submitted into a mixnet-based tallying process. Receipt-freeness is preserved as long as at least one honest rerandomized share b'_i is used in the reconstruction of C .

Limitations. The protocol does not ensure that C is a valid vote encryption, meaning a malicious C could encode an invalid vote, that, upon decryption, leaks unintended information and potentially violates violating RF. While TREnc provides strong privacy guarantees, it offers limited support when it comes to verifying election-specific constraints on the encrypted vote, which typically require non-linear proofs. For instance, proving a vote v is a bit requires a quadratic relation, $v(1 - v) = 0$, requiring a non-linear proof (see, e.g., [9]).

Although NIZK proofs can, in principle, support such constraints, integrating them into this setting with the secret sharing and rerandomization is non-trivial. In particular, a non-linear proof constructed before randomization would not survive the transformation, and constructing such a proof after randomization is infeasible for the voter, who lacks control over the ciphertext randomness. In a multi-randomizer setting, where multiple servers independently rerandomize different shares of a ballot, adapting a non-linear proof locally is also challenging. The transformation applied by each server generally depends on the randomness chosen by others, rendering any isolated adaptation incorrect or unverifiable. As a result, constructing such proofs typically requires interaction between the voter and randomizing parties, which could introduce complexity and security risks, as malicious servers could collude with voters. To address these issues, we introduce a technique in the next section to enforce non-linear constraints on encrypted votes without requiring any voter-randomizer coordination.

5 Voting Schemes

We propose a new approach that avoids requiring voters to construct complex zero-knowledge proofs at ballot submission time. Instead, we shift the validity checking to a dedicated *pre-tally phase*, which takes place after the final combined ciphertexts $\mathbf{C}$ have been reconstructed (see Figure 1), but before tallying begins.

At a high level, our scheme preserves the overall voting flow of Doan et al. from the voter’s perspective. The key difference lies in how the reconstructed ciphertexts $\mathbf{C}$ are handled. Rather than passing them directly to a mixnet, they are first processed in a multi-party computation (MPC) protocol that verifies whether each ciphertext encodes a valid vote. This verification is performed collaboratively by the talliers or those who hold shares of the decryption key without revealing the vote or any auxiliary information. Only ciphertexts that pass this validation step are forwarded to the standard tallying process.

In the following, we present two concrete instantiations of this paradigm: **MiniMix** and **HomoRand**. These schemes differ in the inputs provided by voters to the pre-tally phase and the way the underlying MPC protocol operates. Before delving into the designs of **MiniMix** (Section 5.2) and **HomoRand** (Section 5.3), we first outline their shared structure in Section 5.1, following the general voting system definition in Definition 4.1. We then give the correctness and the security theorem statements of the constructions in Section 6.

5.1 Overview

Key Generation. The election authority EA initiates the setup by running the **SetupElection** algorithm, generating a public/secret key pair (PK, SK) . The public key PK is posted on a public bulletin board PBB, while the private key SK is shared among T talliers using a threshold decryption scheme. Key generation can be distributed securely in prime-order groups through standard techniques.

Voting Phase. To cast a vote $\mathbf{v}$, a voter executes the **Vote** algorithm. In principle, this procedure begins by secret-sharing $\mathbf{v}$ into n shares using a t -out-of- n threshold secret sharing scheme (Section 2). Each share is then independently encrypted under a **TREnc** (Section 2), producing ballot shares $\{\mathbf{b}_i\}_{i \in [n]}$. The complete ballot $\mathbf{b}$ consists of this set of encrypted shares.

Each $\mathbf{b}_i$ is then submitted to a distinct randomizing server, while the corresponding trace $\tau = \text{TraceBallot}(\mathbf{b})$ is simultaneously transmitted to PBB. Upon receiving a $\mathbf{b}_i$, a server performs a local verification using the validity checks defined by **TREnc**, optionally including a zero-knowledge proof verification. It further ensures that no duplicate traces are present with respect to all traces on PBB. Valid ballot shares are re-randomized using **ProcessBallot**($\mathbf{b}_i$) and made available on the bulletin board. Voters can later verify inclusion of their vote by querying **VerifyVote** on the trace τ and confirming its appearance on PBB.

Tallying Phase. After voting concludes, the tallying phase begins. Talliers aggregate ballot shares by comparing the traces on PBB with the complete traces τ posted by each voter. The **Valid** function checks whether at least t valid shares

are present for each ballot. If the condition is met, the CPA components are extracted from the corresponding shares and combined using Lagrange interpolation. Owing to the linearity of the interpolation, this process can be performed directly in the encrypted domain, yielding a final ciphertext that encrypts the original vote. Ballots failing the **Valid** check are discarded.

As noted by Doan et al. [10], this reconstruction step is critical for mitigating adversarial influence. A malicious voter may attempt to deviate from the protocol by submitting malformed or inconsistent shares. However, since all valid ballot shares on the bulletin board (possibly more than t) are combined, only one message can be reconstructed. Importantly, the adversary cannot anticipate which subset of shares will be successfully posted (e.g., due to the random failure of non-operational randomizers). Consequently, to ensure that the tally reflects their intended vote, even adversarial voters are incentivized to submit a consistent and correct ballot.

Finally, the reconstructed ciphertexts corresponding to valid ballots are submitted to the **PreTally** function, which checks that the encrypted vote is within the intended range. Depending on its inputs, the **PreTally** function behaves differently. In the **MiniMix** construction (Figure 2), which utilizes the MPC-**MiniMix** protocol from Algorithm 1, the voter submits only the encrypted vote. In contrast, the **HomoRand** construction uses the MPC-**HomoRand** protocol from Algorithms 2 and 3, where the voter also sends additional values to assist the participating parties in verifying the validity of the encrypted vote. Depending on the deployment scenario, the participating parties may include the talliers or, alternatively, external authorities. For simplicity, we assume that the talliers are responsible for performing this validation in the current setting. Invalid ballots are discarded based on the outcome of the **PreTally** check. The talliers then run the **Tally** function to compute the election result r based on the valid ones according to the election rules. A proof of correctness Π is generated and can be verified by anyone using the **VerifyResult** algorithm. In the instantiations below, the number of servers n and the threshold t are implicit inputs for all algorithms.

Relationship between correctness and receipt-freeness thresholds. Our constructions rely on a secret sharing scheme where at least t valid ballot shares are needed to reconstruct a vote. This implies that the system can tolerate up to $n - t$ missing or invalid shares without affecting correctness. On the privacy side, receipt-freeness holds as long as the reconstruction includes at least one honestly rerandomized share. So even if up to $t - 1$ ballot shares are maliciously processed, it is infeasible for the adversary to compromise voter privacy. In other words, the system remains secure against up to $t - 1$ compromised processing servers, which matches the tight bound shown in prior work by Doan et al. [10].

5.2 The MiniMix Construction

In this scheme, the input to the **PreTally** is a ballot $\mathbf{b} = \{\mathbf{b}_i\}_{i \in [n]}$ randomized by randomizers, where each $\mathbf{b}_i$ is a **TREnc**'s encryption of a share x_i of the voter's intended message v .

To initiate the **PreTally** procedure, the talliers (and optionally the public) verify each ballot share $\mathbf{b}_i$ by applying the **TREnc.Ver** function (see Section 2). A ballot $\mathbf{b}$ is deemed valid by **Valid**(BB, $\mathbf{b}$) (see Figure 2) if at least t valid shares are posted on the public bulletin board. Upon successful validation, the combination algorithm **Combine** is publicly executed by any of the talliers. It takes as input the homomorphic components of all available valid ballot shares—up to n shares—and outputs a combined ciphertext $\mathbf{C}_0$, representing the encrypted vote $\mathbf{v}$. Due to the properties of Lagrange interpolation, reconstructing with more than t valid shares preserves the correctness of the resulting ciphertext.

Subsequently, T talliers jointly executes the MPC-MiniMix as described in Algorithm 1 with the input $\mathbf{C} = \{\mathbf{C}_j\}_{j \in [d]}$, where $\mathbf{C}_j = \mathbf{C}_0 \cdot \text{CPA}(\text{PK}, -v_j)$. In particular, each tallier $\mathcal{T}_k$ for $k \in [T]$ shuffles the d ciphertexts in $\mathbf{C}$, applying an independent random factor. It also produces a publicly verifiable ZKPoK proof, attesting to correctness. Misbehaving parties are identified through proof verification. After the last tallier has completed their respective rounds, all the talliers jointly decrypt the final output $\mathbf{C}^{(T)} = \{\mathbf{C}_j^{(T)}\}_{j \in [d]}$. As $\mathbf{C}^{(T)}$ is encrypted under the **TREnc**'s PK, decryption proceeds using **TREnc.Dec**(SK, $\cdot$) (Algorithm 1, line 20). The **PreTally** procedure outputs 1 as soon as one of the decrypted ciphertexts equals 0, confirming that the original vote $\mathbf{v}$ belongs to the designated valid range; otherwise, it outputs 0. Invalid votes are discarded, and the **Tally** function is applied to the combined ciphertext $\mathbf{C}_0$ of each valid ballot to compute the final outcome according to the election rules. A formal correctness proof of the MPC-MiniMix is given in Appendix B.1.

Discussion. It is worth noting that the MiniMix construction can be adapted to minimize online-phase computation. In the presented variant, the shuffle is performed after a ballot is available on the PBB, which may lead to inefficiencies such as increased latency and synchronization delays.

As an alternative, the authorities may jointly shuffle the encrypted vote domain $\{\mathbf{V}_j = \text{CPA}(v_j)\}_{j \in [d]}$ under a secret permutation π , yielding a randomized sequence $\{\mathbf{V}'_{\pi(j)}\}_{j \in [d]}$. Given a combined ciphertext $\mathbf{C}_0$ (as defined earlier), they compute $\mathbf{C}'_j = \mathbf{C}_0 / \mathbf{V}'_{\pi(j)}$ for each $j \in [d]$, resulting in ciphertexts encrypting $(\mathbf{v} - v_{\pi(j)})$. These can be tested for equality to zero to verify whether $\mathbf{v}$ belongs to the valid domain. As π remains hidden, the test leaks no information about the actual vote. To avoid linkability, each ballot must be verified against a freshly shuffled domain encryption. Otherwise, repeated shuffles could reveal identical vote positions, enabling correlation across ballots. Hence, the number of domain shuffles must scale with the number of ballots or eligible voters.

5.3 The HomoRand Construction

Due to space constraints, the full specification of **HomoRand** is provided in Appendix A.3; here we give a high-level overview. The input to **PreTally** is a randomized ballot $\mathbf{b} = \{\mathbf{b}_i\}_{i \in [n]}$, where each $\mathbf{b}_i$ now consists of l components $\mathbf{b}_{ji}$ for $j \in [l]$. To this end, the vote $\mathbf{v}$ is first decomposed into an l -bit string $\{b_j\}_{j \in [l]}$, and each bit b_j is shared using t -out-of- n secret sharing scheme to obtain n share $\{b_{ji}\}_{i \in [n]}$. Each share b_{ji} is then encrypted separately under two **TREnc** public

SetupElection (λ) $(SK, PK) \leftarrow \text{TREnc.Gen}(1^\lambda)$ return PK	ProcessBallot (b_i) return $\text{TREnc.Rand}(PK, b_i)$
Vote ($id, v[, \text{aux}]$) $\{x_1, \dots, x_n\} \leftarrow \text{Share}(n, t, v)$ if aux is empty for $i = 1$ to n do $lk_i \leftarrow \text{TREnc.LGen}(PK)$ else $\{lk_i\}_{i=1}^n \leftarrow \text{aux}$ for $i = 1$ to n do $b_i \leftarrow \text{TREnc.LEnc}(PK, lk_i, x_i)$ return $b = \{b_i\}_{i=1}^n$	TraceBallot (b) $\tau_i \leftarrow \text{TREnc.Trace}(b_i)$ return $\tau = \{\tau_i\}_{i=1}^n$
Valid (BB, b) if $\exists b' \in BB \wedge \exists \tau'_i \subset \text{TraceBallot}(b') :$ $\tau'_i \subset \text{TraceBallot}(b)$ then return $\perp$ $k = 0$ for $i = 1$ to $ b $ do if $\text{TREnc.Ver}(PK, b_i) = 1$ then $k \leftarrow k + 1$ if $k \geq t$ then return 1 else return 0	PreTally (BB, SK, b) if $\text{Valid}(BB, b) = 0$ then return 0 for $i = 1$ to $ b $ do $c_i \leftarrow \text{TREnc.Strip}(PK, b_i)$ $C_0 \leftarrow \text{Combine}(n, t, \{c_i\}_{i=1}^{\leq n})$ for $j = 1$ to d do $C_j \leftarrow C_0 \cdot \text{CPA}(PK, -v_j)$ return $\text{MPC-MiniMix}(PK, SK, C = \{C_j\}_{j=1}^d)$
	VerifyVote (PBB, τ) if $\exists b \in PBB : \text{Valid}(b) \wedge \tau == \text{TraceBallot}(b)$ then return 1 else return 0

Fig. 2: MiniMix instantiation of our voting scheme.

keys, yielding a pair of ciphertexts in $\mathbb{G}$ and $\hat{\mathbb{G}}$. A non-interactive randomizable proof (e.g., Groth-Sahai proofs [12]) accompanies each pair, proving consistency across the two encryptions, i.e., demonstrating that they indeed encrypt the same value. As a result, each ballot share contains l components, where each includes two TREnc ciphertexts and a consistency proof. Each ballot share is re-randomized by a randomizer, and the resulting share is posted to the public board.

The **PreTally** protocol begins by validating each ballot b posted on the public bulletin board. This involves applying TREnc.Ver to the ciphertexts and invokes PrfVerify on the associated proof. A ballot is deemed valid if, for every bit index $j \in [d]$, there exist at least t valid shares $\{b_{ji}\}_i$ posted on the board. Once a ballot passes verification, the associated proofs are discarded. Next, the **Combine** algorithm aggregates the homomorphic components of the accepted ballot shares (up to n) to reconstruct two CPA encryptions of $\{b_j\}_{j \in [l]}$ in $\mathbb{G}$ and $\hat{\mathbb{G}}$. The protocol then invokes the **MPC-HomoRand** procedure from Algorithms 2 and 3 to jointly verify that each b_j is a bit.

5.4 Efficiency Discussion

Table 1 compares the computational costs incurred during the casting of a single ballot under the two proposed constructions. All costs reported are per voter, per randomizer, or per tallier, as appropriate.

	SetupElection	Vote	ProcessBallot	PreTally
MiniMix	$1 \cdot \text{TREnc.Gen} \in \mathbb{G}$	$n \cdot \text{TREnc.Enc}$	$1 \cdot \text{TREnc.Rand}$	$\text{shuffle}(d) + d \cdot \text{rand} + d \cdot \text{dec}$
HomoRand	$1 \cdot \text{TREnc.Gen} \in \mathbb{G}$ $1 \cdot \text{TREnc.Gen} \in \hat{\mathbb{G}}$	$2ln \cdot \text{TREnc.Enc}$ $\mathbb{G}^{16ln} \times \hat{\mathbb{G}}^{16ln}$	$2l \cdot \text{TREnc.Rand}$ $\mathbb{G}^{14l} \times \hat{\mathbb{G}}^{14l}$	$2l \cdot \text{rand} + (l + 1) \cdot \text{dec}$

Table 1: Computational cost per ballot under each construction.

Computational Costs. On the voter’s side, the **Vote** algorithm in **MiniMix** requires n invocations of **TREnc.Enc**, as each ballot share is encrypted independently. In contrast, **HomoRand** imposes a higher load: $2ln$ encryptions (two per bit per share) and $16ln$ exponentiations for Groth-Sahai consistency proofs across all $i \in [n]$ and $j \in [l]$ (see Appendix A for details on proof computation). Each randomizer, executing **ProcessBallot**, performs 1 **TREnc** rerandomization in **MiniMix**, while in **HomoRand**, this increases to $2l$, alongside rerandomizing the associated proofs. Talliers executing **PreTally** incur costs primarily from the underlying MPC protocols. In MPC-**MiniMix** (Algorithm 1), each tallier (i) shuffles d ciphertexts with verifiable proofs at a cost denoted $\text{shuffle}(d)$ (lines 4–9); (ii) applies random factors to shuffled ciphertexts with proofs (lines 11–13), amounting to $d \cdot \text{rand}$ operations; and (iii) participates in the joint decryption of d ciphertexts (line 19–24), contributing $d \cdot \text{dec}$ operations. In MPC-**HomoRand**, each tallier performs $2l \cdot \text{rand}$ operations (Algorithm 2, lines 2–8) and engages in $l+1 \cdot \text{dec}$ operations (Algorithm 3, lines 9 and 14).

Verification Costs. Anyone, including external auditors or voters, can verify the **PreTally** result by checking the posted proofs. In **HomoRand**, the verification requires $2l \cdot \text{rand} + (l + 1) \cdot \text{dec}$ while in **MiniMix** it involves $d \cdot (\text{rand} + \text{dec})$, plus $\text{shuffle}(d)$ proofs from each of the T talliers. Since $l = \log_2(d)$, the relative verification cost ratio scales as roughly $d/(2\log_2 d)$ in favor of **HomoRand** for large domains d , though **MiniMix** incurs additional overhead from shuffle proofs, which scale with both d and T .

While **HomoRand** achieves **PreTally** costs that scale with $l = \log_2(d)$, asymptotically outperforming the linear-in- d cost of **MiniMix**, this efficiency comes at the price of substantially higher computational overhead for voters and randomizers. Specifically, the cost of **Vote** in **HomoRand** scales with both n and l , due to bitwise encryption and the generation of Groth-Sahai proofs. In contrast, **MiniMix** imposes minimal and domain-independent client-side costs, making it an attractive choice in settings where lightweight voting is essential and the number of valid vote options is modest. Conversely, **HomoRand** may be preferable in settings where the verification of **PreTally** is a critical concern—such as public audits or large-scale elections—particularly when voter-side computation is not a bottleneck and the domain size d is large enough to outweigh its setup and voting costs.

6 Security of the Voting Schemes

The proposed voting schemes achieve both *threshold receipt-freeness* and *threshold correctness* (Section 4). Due to space constraints, we provide proof sketches here. Formal proofs appear in the Appendix C.

Theorem 6.1 (Threshold Receipt-Freeness). *Let TREnc be verifiable and TCCA-secure, and let the proof systems employed for the pre-tally and for verifying tally correctness be zero-knowledge. Then both constructions achieve threshold receipt-freeness under a t -out-of- n sharing scheme with threshold $t_{\text{rf}} = t - 1$. More precisely, for the MiniMix scheme, $\Pr[\text{Exp}_{\mathcal{A}, \mathcal{V}, t_{\text{rf}}}^{\text{deceive}}(\lambda) = 1] \leq \varepsilon_{\text{verif}}$ and $\text{Adv}_{\mathcal{A}, \mathcal{V}}^{\text{rf}, t_{\text{rf}}, \beta}(1^\lambda) \leq \varepsilon_{\text{ZK}} + q(n - t_{\text{rf}})\varepsilon_{\text{TCCA}}$. For the HomoRand scheme, $\Pr[\text{Exp}_{\mathcal{A}, \mathcal{V}, t_{\text{rf}}}^{\text{deceive}}(\lambda) = 1] \leq \varepsilon_{\text{verif}}$ and $\text{Adv}_{\mathcal{A}, \mathcal{V}}^{\text{rf}, t_{\text{rf}}, \beta}(1^\lambda) \leq \varepsilon_{\text{ZK}} + lq(n - t_{\text{rf}})(2\varepsilon_{\text{TCCA}} + \varepsilon_{\text{SxDH}})$. Here, l is the bit-length of the vote domain, and ε_{ZK} , $\varepsilon_{\text{SxDH}}$, $\varepsilon_{\text{verif}}$, $\varepsilon_{\text{TCCA}}$ bound the adversarial advantage against the ZK proof systems, the SXDH (Symmetric eXternal Diffie-Hellman) assumption, verifiability, and TCCA-security of TREnc, respectively; q is the number of ballot-append queries.*

Proof. We sketch the proof for MiniMix; the case of HomoRand is analogous. Threshold receipt-freeness is defined via two experiments.

In the first, $\text{Exp}_{\mathcal{A}, \mathcal{V}, t_{\text{rf}}}^{\text{deceive}}(\lambda)$ [10], security follows from TREnc’s TCCA security, verifiability, and the correctness of the secret sharing scheme, as shown in [10].

In the second, $\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{rf}, t_{\text{rf}}, \beta}(\lambda)$, the adversary must provide two valid ballots with matching traces. We define a sequence of hybrid games starting from $\beta = 0$ (honest setup) and ending at $\beta = 1$ (simulated setup). Let $\mathcal{A}$ query ballot pairs B_0, B_1 with identical traces to BB_0 and BB_1 respectively. We progressively replace the processed ballot shares of B_0 with those of B_1 in BB_0 , relying on the TCCA security of TREnc to preserve indistinguishability at each step, incurring at most $\varepsilon_{\text{TCCA}}$ advantage per swap. This may introduce inconsistencies in the tally since this changes the underlying plaintext. However, since the vote intent of B_0 is known from TREnc.Dec’s correctness, the zero-knowledge simulator can emulate the decryption step (Algorithm 1, line 20) to match the expected result of pre-tally phase. Thus, $\mathcal{A}$ cannot distinguish whether B_0 or B_1 was processed, even when B_0 encodes an invalid vote, except with an error bounded by ε_{ZK} . A hybrid argument over q queries yields $\text{Adv}_{\mathcal{A}, \mathcal{V}}^{\text{rf}, t_{\text{rf}}, \beta}(1^\lambda) \leq \varepsilon_{\text{ZK}} + q(n - t_{\text{rf}})\varepsilon_{\text{TCCA}}$.

Theorem 6.2 (Threshold Correctness). *Let TREnc be traceable and verifiable, and assume that the underlying NIZK proof systems are correct. Then both constructions achieve threshold correctness with $t_{\text{corr}} = n - t$ under a t -out-of- n secret sharing scheme. More precisely, for any efficient adversary $\mathcal{A}$ making q ballot-append queries, $\Pr[\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{corr}, t_{\text{corr}}}(\lambda) = 1] \leq qn\varepsilon_{\text{trace}} + \varepsilon_{\text{corr}}$ for MiniMix and $\Pr[\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{corr}, t_{\text{corr}}}(\lambda) = 1] \leq 2qln\varepsilon_{\text{trace}} + \varepsilon_{\text{corr}}$ for HomoRand, where l is the vote bit-length, and $\varepsilon_{\text{trace}}$, $\varepsilon_{\text{corr}}$ bound $\mathcal{A}$ ’s advantage against the traceability of TREnc and the correctness error of the underlying MPC protocol, respectively.*

Proof. In the threshold correctness experiment, denoted $\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{corr}, t_{\text{corr}}}(\lambda)$ [10], $\mathcal{A}$ submits q pairs of valid ballots to two election views, each omitting at most

t_{corr} ballot shares. As all ballots derive from honestly generated ones encoded as TREnc ciphertexts, traceability ensures $\mathcal{A}$ cannot alter the vote intent without changing trace values, except with negligible probability. This yields a bound of $qn\varepsilon_{\text{trace}}$ for MiniMix, and $2qln\varepsilon_{\text{trace}}$ for HomoRand. Once ballots are posted to the PBB, the correctness of the secret sharing scheme guarantees that the inputs to the pre-tally phase correctly reflect the original vote intent. The correctness of the underlying NIZK proofs in MPC-MiniMix and MPC-HomoRand then ensures that both views yield identical outputs for valid votes.

Verifiability. Our schemes ensure both *individual* and *universal verifiability*. Since the voter’s voting process in MiniMix follows the protocol of [10], it inherits individual verifiability via the traceability of TREnc. In HomoRand, voters additionally provide consistency proofs across the two TREnc ciphertexts in each ballot share, without affecting traceability. Hence, no adversary can alter the vote intent without detection. For universal verifiability, both constructions employ publicly verifiable ZKPoKs in pre-tally and tally phases. The soundness of these proofs ensures that all accepted ballots encode valid votes and that decryption is performed correctly, thereby guaranteeing a trustworthy tally outcome.

7 Conclusion

We propose a new direction for validating encrypted ballots in threshold receipt-free voting systems, where ballots are non-interactively rerandomized by multiple independent ballot processing servers. Our approach introduces a pre-tally validation phase executed in a multiparty computation style, allowing authorities to verify the validity of encrypted votes without decrypting them or requiring voter-supplied validity proofs.

We develop two constructions that achieve threshold receipt-freeness and verifiability while addressing privacy risks present in prior mixnet-based systems. Both schemes support homomorphic or mixnet-based tallying, depending on vote range constraints, and crucially reveal only the validity status of a ballot, nothing more. To our knowledge, this is the first threshold receipt-free solution to achieve better privacy independently of the tallying technique.

Our efficiency analysis recommends using MiniMix when voter-side efficiency is critical and the number of valid vote options is small or moderate, while HomoRand is better suited for large or complex vote domains. In addition to our core designs, we also discuss how existing validity-check techniques can be adapted to fit our pre-tally framework. An open direction is to reduce the computational and communication complexity of the pre-tally process, or to devise alternative mechanisms such as randomizable validity proofs that can be verified in a single-pass setting, even in the presence of fully malicious randomizers.

Acknowledgments

Thomas Peters is a research associate of the Belgian Fund for Scientific Research (F.R.S.-FNRS). This work has been funded in part by the Walloon Region through the project CyberExcellence (convention number 2110186).

References

1. Bayer, S., Groth, J.: Efficient zero-knowledge argument for correctness of a shuffle. In: Eurocrypt. pp. 263–280. Springer (2012)
2. Benaloh, J., Tuinstra, D.: Receipt-free secret ballot elections. In: Proc. 26th ACM Symp. on Theory of Computing (STOC). pp. 544–553. ACM (1994)
3. Blazy, O., Fuchsbaauer, G., Pointcheval, D., Vergnaud, D.: Signatures on randomizable ciphertexts. In: Public Key Cryptography–PKC. pp. 403–422. Springer (2011)
4. Chaidos, P., Cortier, V., Fuchsbaauer, G., Galindo, D.: BeleniosRF: A non-interactive receipt-free electronic voting scheme. In: Proc. ACM CCS. pp. 1614–1625. ACM (2016)
5. Chaum, D., Pedersen, T.P.: Wallet databases with observers. In: Advances in Cryptology – CRYPTO ’92. pp. 89–105. Springer (1993)
6. Cramer, R., Gennaro, R., Schoenmakers, B.: A secure and optimally efficient multi-authority election scheme. In: EUROCRYPT ’97. pp. 103–118. Springer (1997)
7. Devillez, H., Pereira, O., Peters, T.: Traceable receipt-free encryption. In: Proc. ASIACRYPT 2022. pp. 273–303. Springer (2022)
8. Devillez, H., Pereira, O., Peters, T., Yang, Q.: Can we cast a ballot as intended and be receipt free? In: Proc. IEEE S&P 2024. pp. 3440–3457. IEEE (2024)
9. Doan, T.V.T., Pereira, O., Peters, T.: Encryption mechanisms for receipt-free and perfectly private verifiable elections. In: Proc. ACNS. pp. 257–287. Springer (2024)
10. Doan, T.V.T., Pereira, O., Peters, T.: Threshold receipt-free single-pass evoting. In: Proc. E-Vote-ID 2024. pp. 20–36. Springer (2024)
11. Groth, J., Ostrovsky, R., Sahai, A.: Perfect non-interactive zero knowledge for np. In: Advances in Cryptology-EUROCRYPT 2006. pp. 339–358. Springer (2006)
12. Groth, J., Sahai, A.: Efficient non-interactive proof systems for bilinear groups. In: Proc. EUROCRYPT 2008. pp. 415–432. Springer (2008)
13. Hirt, M.: Multi party computation: Efficient protocols, general adversaries, and voting (2001)
14. McMurtry, E., Pereira, O., Teague, V.: When is a test not a proof? In: Proc. ESORICS 2020. pp. 23–41. Springer (2020)
15. Ozdemir, A., Boneh, D.: Experimenting with collaborative zk-snarks: Zero-knowledge proofs for distributed secrets. In: Proc. USENIX Security 2022. pp. 4291–4308. USENIX Association (2022)
16. Shamir, A.: How to share a secret. Communications of the ACM pp. 612–613 (1979)
17. Terelius, B., Wikström, D.: Proofs of restricted shuffles. In: Africacrypt. pp. 100–113. Springer (2010)

A Details of the HomoRand Construction

A.1 Computational Setting

We rely on an efficient **Setup** algorithm to generate common public parameters $\mathbf{pp}$. Given a security parameter λ , **Setup**(1^λ) outputs $\mathbf{pp} = (\mathbb{G}, \hat{\mathbb{G}}, \mathbb{G}_T, p, e, g, \hat{g})$ where $(\mathbb{G}, \hat{\mathbb{G}}, \mathbb{G}_T)$ are groups of prime order $p > 2^{\text{poly}(\lambda)}$ for some polynomial poly , with $g \leftarrow \mathbb{G}$ and $\hat{g} \leftarrow \hat{\mathbb{G}}$ as generators, and $e : \mathbb{G} \times \hat{\mathbb{G}} \rightarrow \mathbb{G}_T$ as a bilinear map. In this setting, we assume the SXDH assumption, which states that the DDH problem remains computationally hard in both $\mathbb{G}$ and $\hat{\mathbb{G}}$.

A.2 Groth-Sahai Proofs

The Groth-Sahai (GS) proof system [12] provides efficient non-interactive proofs for satisfiability of pairing-product equations. We use them to prove consistency and knowledge of ciphertexts, which supports perfectly rerandomizable proofs and can be efficiently instantiated in pairing-friendly groups under the SXDH assumption. The GS proof system consists of the following PPT algorithms:

- **GSSetup**(1^λ): On input a security parameter λ , outputs a common reference string (CRS) $\text{crs} \in \mathbb{G}_1^4 \times \mathbb{G}_2^4$ for which the commitment scheme is perfectly hiding (or perfectly binding, depending on the mode).
- **GSProve**($\text{crs}, x; w$): On input a CRS crs , a statement x (consisting of a system of pairing-product equations), and a witness w satisfying x , outputs a non-interactive proof π under crs .
- **GSVerify**(crs, x, π): On input a CRS crs , a statement x , and a proof π , returns **accept** if π is a valid proof that x is satisfiable, and **reject** otherwise.
- **GSRand**(crs, x, π): On input a CRS crs , a statement x , and a valid proof π for x , outputs a rerandomized proof π' such that:
 - **GSVerify**(crs, x, π') = **accept**, and
 - π' is computationally indistinguishable from a fresh proof generated by **GSProve**($\text{crs}, x; w$), assuming crs is generated in the perfectly hiding mode.

Following the standard GS notation, we define the map $\iota : \mathbb{G} \rightarrow \mathbb{G}^2$ that maps $X \in \mathbb{G}$ to $\iota(X) = (X, 1)$ and the map $\iota_T : \mathbb{G}_T \rightarrow \mathbb{G}_T^2$ that maps $T \in \mathbb{G}_T$ to $\iota_T(T) = (T, 1)$. We also extend the pairing as $E_1 : \mathbb{G}^2 \times \hat{\mathbb{G}} \rightarrow \mathbb{G}_T^2$ such that $E_1(\mathbf{a}, b) = (e(a_1, b), e(a_2, b))$, $E_2 : \mathbb{G} \times \hat{\mathbb{G}}^2 \rightarrow \mathbb{G}_T^2$ such that $E_2(a, \mathbf{b}) = (e(a_1, b_1), e(a_2, b_1), e(a_1, b_2), e(a_2, b_2))$, and $E : \mathbb{G}^2 \times \hat{\mathbb{G}}^2 \rightarrow \mathbb{G}_T^4$ such that $E(\mathbf{a}, \mathbf{b}) = (e(a_1, b_1), e(a_2, b_1), e(a_1, b_2), e(a_2, b_2))$, where $\mathbf{a} = (a_1, a_2)$ and $\mathbf{b} = (b_1, b_2)$. We use the multiplicative notation for vector space operations.

A.3 The HomoRand Construction

We now detail the construction of the **HomoRand** voting scheme, instantiated using the **TREnc** mechanism from [7]. As it follows the general framework outlined in Section 5.1, we focus here on the technical specification of the scheme's core functions as defined in Figure 3. We assume that it is straightforward to extract specific parts of the **TREnc**'s ciphertext **CT** (as mentioned in Section 2) and public key **PK**. In particular:

- Strip**(**PK**, **CT**): Returns the CPA-encryption component of **CT**. For instance, in **TREnc** [7], this component is $\mathbf{c} = (c_0, c_1, c_2) = (G^m f^\theta, g^\theta, h^\theta)$, where m is the message, $\theta \leftarrow \mathbb{Z}_p$, and $(G, f, g, h) \in \text{PK}$.
- Strip**(**PK**): Extracts the public parameters to compute the CPA part. For example, in **TREnc** [7], we have **Strip**(**PK**) = (G, f, g, h) , where $f = g^\alpha h^\beta$ for secret key **SK** = (α, β) .

SetupElection (λ)	Valid (BB, b)
$(SK_1, PK_1) \leftarrow \text{TREnc.Gen}(1^\lambda)$ $(SK_2, PK_2) \leftarrow \text{TREnc.Gen}(1^\lambda)$ $\sigma \leftarrow \text{PrfSetup}(1^\lambda); \text{crs} \leftarrow \text{GSSetup}(1^\lambda)$ $SK = (SK_1, SK_2)$ return $PK = (PK_1, PK_2, \sigma, \text{crs})$	if $\exists b' \in BB \wedge \exists \tau'_i \subset \text{TraceBallot}(b')$: $\tau'_i \subset \text{TraceBallot}(b)$ then return $\perp$ for $j = 1$ to l do $k = 0$ for $i = 1$ to $ b $ do if $\text{GSVerify}(PK, \theta_{ji}) = 1$ $\wedge \text{TREnc.Ver}(PK_1, C_{ji}) = 1$ $\wedge \text{TREnc.Ver}(PK_2, \hat{C}_{ji}) = 1$ then $k \leftarrow k + 1$ if $k < t$ then return 0 return 1
Vote (id, v, l, [, aux])	TraceBallot (b)
$v = \sum_j b_j 2^j$ for $b_j \in \{0, 1\}, j \in [l]$ for $j = 1$ to l do $\{b_{ji}\}_{i=1}^n \leftarrow \text{Share}(PK, t, b_j)$ if aux is empty for $j = 1$ to l do for $i = 1$ to n do $\text{lk}_{ji} \leftarrow \$ \text{TREnc.LGen}(PK)$ $\hat{\text{lk}}_{ji} \leftarrow \$ \text{TREnc.LGen}(PK)$ else $\{\text{lk}_{ji}, \hat{\text{lk}}_{ji}\}_{j,i} \leftarrow \text{aux}$ for $j = 1$ to l do for $i = 1$ to n do $C_{ji} \leftarrow \text{TREnc.LEnc}(PK_1, \text{lk}_{ji}, b_{ji})$ $\hat{C}_{ji} \leftarrow \text{TREnc.LEnc}(PK_2, \hat{\text{lk}}_{ji}, b_{ji})$ $\theta_{ji} \leftarrow \text{GSProve}(PK, C_{ji}, \hat{C}_{ji})$ $b_{ji} = (C_{ji}, \hat{C}_{ji}, \theta_{ji})$ return $b = \{b_{ji}\}_{j,i}^{l,n}$	$\tau_{ji} \leftarrow (\text{TREnc.Trace}(C_{ji}), \text{TREnc.Trace}(\hat{C}_{ji}))$ return $\tau = \{\tau_{ji}\}_{j,i}^{l,n}$
ProcessBallot (b_i)	PreTally (BB, SK, b)
for $j = 1$ to l do $C'_{ji} \leftarrow \text{TREnc.Rand}(PK_1, C_{ji})$ $\hat{C}'_{ji} \leftarrow \text{TREnc.Rand}(PK_2, \hat{C}_{ji})$ $\theta'_{ji} \leftarrow \text{GSRand}(PK, \hat{C}_{ji}, C_{ji}, \theta_{ji})$ $b_{ji} = (C'_{ji}, \hat{C}'_{ji}, \theta'_{ji})$ return $b'_i = \{b_{ji}\}_{j \in [l]}$	if $\text{Valid}(BB, b) = 0$ then return 0 for $j = 1$ to l do for $i = 1$ to $ b $ do $c_{ji} \leftarrow \text{TREnc.Strip}(PK_1, C_{ji})$ $\hat{c}_{ji} \leftarrow \text{TREnc.Strip}(PK_2, \hat{C}_{ji})$ $c_j \leftarrow \text{Combine}(n, t, \{c_{ji}\}_{i=1}^n)$ $\hat{c}_j \leftarrow \text{Combine}(n, t, \{\hat{c}_{ji}\}_{i=1}^n)$ $C = \{(c_j, \hat{c}_j)\}_{j=1}^l$ return $\text{MPC-HomoRand}(PK, SK, C)$
	VerifyVote (PBB, τ)
	if $\exists b \in \text{PBB} : \text{Valid}(b) \wedge \tau == \text{TraceBallot}(b)$ then return 1 else return 0

Fig. 3: HomoRand instantiation of our voting scheme.

SetupElection(λ): Chooses bilinear groups $(\mathbb{G}, \hat{\mathbb{G}}, \mathbb{G}_{\mathcal{T}})$ of prime order p ($p > 2^{\text{poly}(\lambda)}$) together with $G \leftarrow \$ \mathbb{G}, \hat{G} \leftarrow \$ \hat{\mathbb{G}}$, and

1. Run $\text{TREnc.Gen}(1^\lambda)$ to generate two key pairs (SK_1, PK_1) and (SK_2, PK_2) to encrypt messages in $\mathbb{G}$ and $\hat{\mathbb{G}}$ respectively.
2. Run $\text{GSSetup}(1^\lambda)$ to generate two tuples of 4 random group elements $(\text{crs}_1, \text{crs}_2) \leftarrow \$ \mathbb{G}^4 \times \hat{\mathbb{G}}^4$ such that $\text{crs}_1 = (\mathbf{u}_1, \mathbf{u}_2)$ is seen as a Groth-Sahai CRS to commit to group elements over $\mathbb{G}$ and $\text{crs}_1 = (\varphi, \psi)$ is seen as a Groth-Sahai CRS to commit to group elements over $\hat{\mathbb{G}}$.

The private key consists of $SK = \{SK_1, SK_2\}$ and the public key $PK = \{G, \hat{G}, \text{crs}_1, \text{crs}_2, PK_1, PK_2\}$.

Vote(id, v, l, aux) To encrypt a vote v, a voter presents it as a l -bit string $b_1 b_2 \dots b_l$, then conducts the following steps of $\text{Enc}(PK, b, \text{aux})$ to encrypt each bit $b \in \{b_1, b_2, \dots, b_l\}$:

1. Run $\text{Share}(n, t, b)$: Apply the (t, n) - secret sharing scheme to output n shares $\{s_i\}_{i=1}^n$.
2. Run TREnc.Enc :
 - If aux is specified, set $\text{lk}_{ki} = \text{aux}_{ki}$ for $k = \{1, 2\}, i \in [n]$. Otherwise, $\text{lk}_{ki} = \text{TREnc.LGen}(\text{PK}_k)$.
 - Compute $\mathcal{C}_i = \text{TREnc.LEnc}(\text{PK}_1, \text{lk}_{1i}, G^{s_i})$, $\hat{\mathcal{C}}_i = \text{TREnc.LEnc}(\text{PK}_2, \text{lk}_{2i}, \hat{G}^{s_i})$, where the stripped CPA parts respectively are given by $\mathbf{c}_i = \text{TREnc.Strip}(\text{PK}_1, \mathcal{C}_i)$, $\hat{\mathbf{c}}_i = \text{TREnc.Strip}(\text{PK}_2, \hat{\mathcal{C}}_i)$ such that

$$\mathbf{c}_i = (c_{0i}, c_{1i}, c_{2i}) = (G^{s_i} f^{\alpha_i}, g^{\alpha_i}, h^{\alpha_i})$$

$$\hat{\mathbf{c}}_i = (\hat{c}_{0i}, \hat{c}_{1i}, \hat{c}_{2i}) = (\hat{G}^{s_i} \hat{f}^{\beta_i}, \hat{g}^{\beta_i}, \hat{h}^{\beta_i})$$

with $(f, g, h) = \text{TREnc.Strip}(\text{PK}_1)$, $(\hat{f}, \hat{g}, \hat{h}) = \text{TREnc.Strip}(\text{PK}_2)$, and $\alpha_i, \beta_i \leftarrow \mathbb{Z}_p$.

3. Ensure encrypted messages $G^{s_i}, \hat{G}^{s_i}$ share the same exponent by computing the following for each $i = 1, \dots, n$:
 - Commit to the scalars s_i, α_i in $\mathcal{C}_i$ by $\hat{\mathcal{C}}_{2,i} = \varphi^{s_i} \psi^{\rho_{2i}}, \hat{\mathcal{C}}_{\alpha,i} = \varphi^{\alpha_i} \psi^{\rho_{\alpha i}}$ with $\rho_{2i}, \rho_{\alpha i} \leftarrow \mathbb{Z}_p$.
 - Commit to the scalars s_i, β_i in $\hat{\mathcal{C}}_i$ by $\mathbf{C}_{1,i} = \mathbf{u}_1^{s_i} \mathbf{u}_2^{\rho_{1i}}, \mathbf{C}_{\beta,i} = \mathbf{u}_1^{\beta_i} \mathbf{u}_2^{\rho_{\beta i}}$, with $\rho_{1i}, \rho_{\beta i} \leftarrow \mathbb{Z}_p$.
 - Pick $t \leftarrow \mathbb{Z}_p$ and compute the GS proofs $\boldsymbol{\theta}_i = \{\theta_{ji}\}_{j=1}^8 = (G^{\rho_{2i}} f^{\rho_{\alpha i}}, g^{\rho_{\alpha i}}, h^{\rho_{\alpha i}}, \hat{G}^{\rho_{1i}} \hat{f}^{\rho_{\beta i}}, \hat{g}^{\rho_{\beta i}}, \hat{h}^{\rho_{\beta i}}, \varphi^{\rho_{1i}} \psi^t, \mathbf{u}_1^{\rho_{2i}} \mathbf{u}_2^t)$:

$$E_2(c_{0i}, \varphi) E_2(\theta_{1i}, \psi) = E_2(G, \hat{\mathcal{C}}_{2,i}) E_2(f, \hat{\mathcal{C}}_{\alpha,i}) \quad (1)$$

$$E_2(c_{1i}, \varphi) E_2(\theta_{2i}, \psi) = E_2(g, \hat{\mathcal{C}}_{\alpha,i}) \quad (2)$$

$$E_2(c_{2i}, \varphi) E_2(\theta_{3i}, \psi) = E_2(h, \hat{\mathcal{C}}_{\alpha,i}) \quad (3)$$

$$E_1(\mathbf{u}_1, \hat{c}_{0i}) E_1(\mathbf{u}_2, \theta_{4i}) = E_1(\mathbf{C}_{1,i}, \hat{G}) E_1(\mathbf{C}_{\beta,i}, \hat{f}) \quad (4)$$

$$E_1(\mathbf{u}_1, \hat{c}_{1i}) E_1(\mathbf{u}_2, \theta_{5i}) = E_1(\mathbf{C}_{\beta,i}, \hat{g}) \quad (5)$$

$$E_1(\mathbf{u}_1, \hat{c}_{2i}) E_1(\mathbf{u}_2, \theta_{6i}) = E_1(\mathbf{C}_{\beta,i}, \hat{h}) \quad (6)$$

$$E(\mathbf{u}_1, \hat{\mathcal{C}}_{2,i}) E(\mathbf{u}_2, \boldsymbol{\theta}_{7i}) = E(\mathbf{C}_{1,i}, \varphi) E(\boldsymbol{\theta}_{8i}, \psi) \quad (7)$$

4. Set $(\mathcal{C}_i, \hat{\mathcal{C}}_i, \hat{\mathcal{C}}_{2,i}, \hat{\mathcal{C}}_{\alpha,i}, \mathbf{C}_{1,i}, \mathbf{C}_{\beta,i}, \boldsymbol{\theta}_i)$ as output of $\text{Enc}(\text{PK}, b, \text{aux})$. Denote $\text{CT}_j = \text{Enc}(\text{PK}, b_j, \text{aux}_j)$ for $\text{aux}_j \in \text{aux}$, the voter sends the ciphertext $\text{CT} = \{\text{CT}_j\}_{j=1}^l$ to corresponding randomizers, where $\text{CT}_j = \{\text{CT}_{ji}\}_{i=1}^n$ and $\text{CT}_{ji} = (\mathcal{C}_{ji}, \hat{\mathcal{C}}_{ji}, \hat{\mathcal{C}}_{2,ji}, \hat{\mathcal{C}}_{\alpha,ji}, \mathbf{C}_{1,ji}, \mathbf{C}_{\beta,ji}, \boldsymbol{\theta}_{ji})$ as previously described.

ProcessBallot(PK, CT_i): If PK or $\text{CT}_i = \{\text{CT}_{ji}\}_{j \in [l]}$ do not parse properly, abort.

- Otherwise, a randomizer conducts the following steps for any $j \in [l]$:
- Compute $\mathcal{C}'_{ji} = \text{TREnc.Rand}(\text{PK}_1, \mathcal{C}_{ji})$ with $\mathbf{c}'_{ji} = \text{Strip}(\text{PK}_1, \mathcal{C}'_{ji})$ and $\hat{\mathcal{C}}'_{ji} = \text{TREnc.Rand}(\text{PK}_2, \hat{\mathcal{C}}_{ji})$, where $\hat{\mathbf{c}}'_{ji} = \text{Strip}(\text{PK}_2, \hat{\mathcal{C}}'_{ji})$ such that

$$\mathbf{c}'_{ji} = (c'_{0ji}, c'_{1ji}, c'_{2ji}) = (c_{0ji} \cdot f^{\alpha'_{ji}}, c_{1ji} \cdot g^{\alpha'_{ji}}, c_{2ji} \cdot h^{\alpha'_{ji}})$$

$$\hat{\mathbf{c}}'_{ji} = (\hat{c}'_{0ji}, \hat{c}'_{1ji}, \hat{c}'_{2ji}) = (\hat{c}_{0ji} \cdot \hat{f}^{\beta'_{ji}}, \hat{c}_{1ji} \cdot \hat{g}^{\beta'_{ji}}, \hat{c}_{2ji} \cdot \hat{h}^{\beta'_{ji}}),$$

with $\alpha'_{ji}, \beta'_{ji} \leftarrow \mathbb{Z}_p$.

- Pick $\rho'_{1ji}, \rho'_{\beta ji}, \rho'_{2ji}, \rho'_{\alpha ji}, t'_{ji} \leftarrow \mathbb{Z}_p$ and adapt the GS commitments and proofs:
 - Compute $\hat{C}'_{2,ji} = \hat{C}_{2,ji} \cdot \psi^{\rho'_{2ji}}, \hat{C}'_{\alpha,ji} = \hat{C}_{\alpha,ji} \cdot \varphi^{\alpha'_{ji}} \psi^{\rho_{\alpha ji'}}$, $C'_{1,ji} = C_{1,ji} \cdot u_2^{\rho'_{1ji}}$, and $C'_{\beta,ji} = C_{\beta,ji} \cdot u_1^{\beta'_{ji}} u_2^{\rho'_{\beta ji}}$.
 - Update the proofs $\theta'_{ji} = (\theta_{1ji} \cdot G^{\rho'_{2ji}} f^{\rho_{\alpha ji}}, \theta_{2ji} \cdot g^{\rho_{\alpha ji}}, \theta_{3ji} \cdot h^{\rho_{\alpha ji}}, \theta_{4ji} \cdot \hat{G}^{\rho'_{1ji}} \hat{f}^{\rho_{\beta ji}}, \theta_{6ji} \cdot \hat{g}^{\rho_{\beta ji}}, \theta_{7ji} \cdot \hat{h}^{\rho_{\beta ji}}, \theta_{8ji} \cdot \varphi^{\rho'_{1ji}} \psi^{t'_{ji}}, \theta_{9ji} \cdot u_1^{\rho'_{2ji}} u_2^{t'_{ji}})$.
- Publish $CT'_{ji} = (C'_{ji}, \hat{C}'_{ji}, \hat{C}'_{2,ji}, \hat{C}'_{\alpha,ji}, C'_{1,ji}, C'_{\beta,ji}, \theta'_{ji})$.

Valid(PK, CT'): Abort and return 0 if PK or CT' is not parsed properly. Return 1 if there exists a subset $I \subset [n]$ such that $|I| \geq t$ and for all $i \in I$:

- $\text{TREnc.Ver}(\text{PK}_1, C'_{ji}) = 1$ and $\text{TREnc.Ver}(\text{PK}_2, \hat{C}'_{ji}) = 1$, and
- The equations 1- 7 hold.

Otherwise, return 0. If $\text{Valid}(\text{PK}, \text{CT}') = 1$, update $\text{CT}' \leftarrow \{\text{CT}'_{ji}\}_{j,i}$ for $i \in I$ and all $j \in [l]$.

PreTally(SK, CT'): Abort and output 0 if PK or CT' does not parse properly or $\text{Valid}(\text{PK}, \text{CT}') = 0$. Otherwise, conduct the following steps:

1. For each $\text{CT}'_{ji} \in \text{CT}'$, run $\text{Strip}(\text{PK}, (C'_{ji}, \hat{C}'_{ji}))$ to only extract the CPA components, denoted as $\text{CPA}(\text{CT}'_{ji})$.
2. Run $\text{Combine}(n, t, \{\text{CPA}(\text{CT}'_{ji})\}_{i=1}^{|I|})$ using Lagrange interpolation for all $j \in [l]$. Since the CPA parts in TREnc is homomorphic, this results in $(c'_j, \hat{c}'_j)$, where

$$\begin{aligned} c'_j &= (c'_{0j}, c'_{1j}, c'_{2j}) = (G^{b_j} f^{\alpha_j + \alpha'_j}, g^{\alpha_j + \alpha'_j}, h^{\alpha_j + \alpha'_j}) \\ \hat{c}'_j &= (\hat{c}'_{0j}, \hat{c}'_{1j}, \hat{c}'_{2j}) = (\hat{G}^{b_j} \hat{f}^{\beta_j + \beta'_j}, \hat{g}^{\beta_j + \beta'_j}, \hat{h}^{\beta_j + \beta'_j}), \end{aligned}$$

where $\alpha_j + \alpha'_j = \sum_{i=1}^{|I|} \lambda_{ji}(\alpha_{ji} + \alpha'_{ji})$ and $\beta_j + \beta'_j = \sum_{i=1}^{|I|} \lambda_{ji}(\beta_{ji} + \beta'_{ji})$. Set $C' = \{C'_j\}_{j=1}^l$ where $C'_j = (c'_j, \hat{c}'_j)$.

3. Run $\text{MPC-HomoRand}(\text{PK}, C')$ (Algorithms 2 and 3) as described in Section B.2.

PreTally returns 1 if and only if v is valid. Finally, the talliers (or anyone) compute the ciphertext $\text{ct} = \prod_{j=1}^l (c'_j)^{2^j}$, and forwarded for tallying according to the standard procedure.

B MPC protocols' Correctness

B.1 MPC-MiniMix

Theorem B.1. *Assuming all parties act honestly, the MPC-MiniMix protocol (Algorithm 1) returns 1 if and only if the encrypted message m lies within the designated valid domain $\mathcal{V} = \{v_1, \dots, v_d\} \subset \mathbb{Z}_p$.*

Proof. The protocol takes as input a vector of ciphertexts $\mathbf{C} = \{\mathbf{C}_j\}_{j \in [d]}$, where each $\mathbf{C}_j = \text{CPA}(\text{PK}, m - v_j)$. Let $P_{k,\dots,T}$ denote the composition of permutations $P_k \circ P_{k+1} \circ \dots \circ P_T$, and let $P := P_1 \circ \dots \circ P_T$ denote the global permutation applied to the ciphertexts. Each party $\mathcal{T}_k$ (for $k \in [T]$) performs two operations:

1. It applies a secret shuffle P_k to permute the ciphertexts (lines 5–10).
2. It re-randomizes each ciphertext by exponentiating it with fresh non-zero randomness $\alpha_j^{(k)}$ (lines 11–15).

Each party also produces zero-knowledge proofs (ZKPoKs) demonstrating the correctness of the shuffle and the exponentiation. Since all parties are honest, all such proofs (lines 10 and 14) are valid, and verification at line 17 succeeds. Thus, the system proceeds without aborts, and the transformed ciphertexts $\mathbf{C}^{(k)}$ are passed to the next party.

After all T rounds, the final ciphertext vector $\mathbf{C}^{(T)}$ satisfies:

$$\mathbf{C}_{P(j)}^{(T)} = \text{CPA} \left(\text{PK}, (m - v_j) \cdot \prod_{k=1}^T \alpha_{P_k, \dots, T(j)}^{(k)} \right),$$

where all $\alpha_{P_k, \dots, T(j)}^{(k)} \in \mathbb{Z}_p^*$ due to honest randomness generation. (We omit explicit randomness from the notation for clarity.)

In the final step (lines 21–26), all talliers jointly decrypt each ciphertext using a threshold decryption procedure. Given correctness of the decryption scheme and the honesty of all parties, this step reliably reveals the plaintext of each ciphertext.

If $m \in \mathcal{V}$, then for some $j \in [d]$, we have $m - v_j = 0$, and so $\text{Dec}(\text{SK}, \mathbf{C}_{P(j)}^{(T)})$ decrypts to 0, despite any multiplicative randomization. Conversely, if $m \notin \mathcal{V}$, then $m - v_j \neq 0$ for all j . Since each ciphertext is multiplicatively re-randomized by a non-zero scalar $\alpha_{P_k, \dots, T(j)}^{(k)}$, $\mathbf{C}_{P(j)}^{(T)}$ decrypts to a non-zero value. Thus, the protocol outputs 1 if and only if $m \in \mathcal{V}$, as claimed.

B.2 MPC-HomoRand

Theorem B.2. *Assuming all parties act honestly, and instantiating the CPA encryption as described in Section A.3, the MPC-HomoRand protocol (Algorithms 2 and 3) outputs 1 if and only if the encrypted message m lies within the designated valid domain $\mathcal{V} = \{0, \dots, 2^l - 1\}$.*

Proof. Following the HomaRand instantiation described in Section A.3, the inputs to Algorithm 2 are ciphertext pairs $\mathbf{C}' = \{(\mathbf{c}'_j, \hat{\mathbf{c}}'_j)\}_{j \in [l]}$. For notational simplicity, we represent each ciphertext as $\mathbf{c}'_j = (G^{b_j} f^{\alpha_j}, g^{\alpha_j}, h^{\alpha_j})$ and $\hat{\mathbf{c}}'_j = (\hat{G}^{b_j} \hat{f}^{\beta_j}, \hat{g}^{\beta_j}, \hat{h}^{\beta_j})$ for $\alpha_j, \beta_j \in \mathbb{Z}_p$. Each $\mathbf{c}'_j$ and $\hat{\mathbf{c}}'_j$ encrypts the j -th bit b_j of the underlying message m under independent public keys PK_1 and PK_2 respectively.

Each party $\mathcal{T}_k$ (for $k \in [T]$) independently rerandomizes $\mathbf{c}'_j$ and $\hat{\mathbf{c}}'_j$ (lines 4–5) for random exponents $\gamma_{k,j}, \lambda_{k,j}, r_{k,j}, s_{k,j} \leftarrow \mathbb{Z}_p \setminus \{0\}$. Explicitly, we have:

$$\begin{aligned}\mathbf{c}''_{k,j} &= (G^{b_j \gamma_{k,j} + \lambda_{k,j}}, 1, 1) \cdot (f^{\gamma_{k,j} \alpha_j + r_{k,j}}, g^{\gamma_{k,j} \alpha_j + r_{k,j}}, h^{\gamma_{k,j} \alpha_j + r_{k,j}}), \\ \hat{\mathbf{c}}''_{k,j} &= (\hat{G}^{(1-b_j) \lambda_{k,j}}, 1, 1) \cdot (\hat{f}^{s_{k,j} - \beta_j \lambda_{k,j}}, \hat{g}^{s_{k,j} - \beta_j \lambda_{k,j}}, \hat{h}^{s_{k,j} - \beta_j \lambda_{k,j}}).\end{aligned}$$

Each randomized ciphertext is accompanied by a ZKPoK $\pi_{k,j}$ attesting to the correctness of the transformation, and due to the honesty of all parties, all proofs are valid and accepted in Algorithm 3 (line 4).

The next step involves publicly aggregating the rerandomized ciphertexts (lines 8–9), which can be done by any party of them and verified by any party:

$$\begin{aligned}\mathbf{c}''_j &= \prod_{k \in [T]} \mathbf{c}''_{k,j} = (G^{b_j \gamma_j + \lambda_j}, 1, 1) \cdot (f^{\alpha_j \gamma_j + r_j}, g^{\alpha_j \gamma_j + r_j}, h^{\alpha_j \gamma_j + r_j}), \\ \hat{\mathbf{c}}''_j &= \prod_{k \in [T]} \hat{\mathbf{c}}''_{k,j} = (\hat{G}^{(1-b_j) \lambda_j}, 1, 1) \cdot (\hat{f}^{s_j - \beta_j \lambda_j}, \hat{g}^{s_j - \beta_j \lambda_j}, \hat{h}^{s_j - \beta_j \lambda_j}),\end{aligned}$$

where $\gamma_j = \sum_k \gamma_{k,j}$, $\lambda_j = \sum_k \lambda_{k,j}$, and similarly for r_j, s_j .

Decryption of $\mathbf{c}''_j$ (line 10) yields:

$$X_j = G^{b_j \gamma_j + \lambda_j},$$

since all parties are honest and the decryption is correct.

Next, the parties compute:

$$\bar{\mathbf{c}}'_j := \text{CPA}(\text{PK}_2, \hat{G}; 0) / \hat{\mathbf{c}}'_j,$$

and evaluate the pairings:

$$\begin{aligned}x_j &= \frac{e(X_j, \bar{\mathbf{c}}'_{0j})}{e(G, \hat{\mathbf{c}}''_{0j})} = e(G, \hat{G})^{b_j(1-b_j)\gamma_j} \cdot e(G, \hat{f})^{-\beta_j b_j \gamma_j - s_j}, \\ y_j &= \frac{e(X_j, \bar{\mathbf{c}}'_{1j})}{e(G, \hat{\mathbf{c}}''_{1j})} = e(G, \hat{g})^{-\beta_j b_j \gamma_j - s_j}, \\ z_j &= \frac{e(X_j, \bar{\mathbf{c}}'_{2j})}{e(G, \hat{\mathbf{c}}''_{2j})} = e(G, \hat{h})^{-\beta_j b_j \gamma_j - s_j}.\end{aligned}$$

In the final step (line 17), the parties jointly decrypt under SK_2 to obtain

$$Y = \prod_{j \in [l]} e(G, \hat{G})^{b_j(1-b_j)\gamma_j},$$

Observe that $b_j(1-b_j) = 0$ if and only if $b_j \in \{0, 1\}$, and $\gamma_j \neq 0$ for all j by the soundness of the zero-knowledge proofs. Hence, $Y = 1_{\mathbb{G}_T}$ if and only if all b_j are bits. In that case, $m = \sum_j b_j 2^j \in \mathcal{V}$, and the protocol returns 1.

Conversely, if $m \notin \mathcal{V}$, then there exists some j^* such that $b_{j^*} \notin \{0, 1\}$, leading to $b_{j^*}(1-b_{j^*})\gamma_{j^*} \neq 0$ and thus $Y \neq 1_{\mathbb{G}_T}$. In this case, the protocol returns 0, as desired.

$\mathcal{O}\text{init}(\lambda)$ $(pk, sk) \leftarrow \text{SetupElection}(1^\lambda)$ $BB_0 \leftarrow \perp; BB_1 \leftarrow \perp; \mathcal{L} \leftarrow \perp$ return (pk, sk)	$\mathcal{O}\text{vote}(id, v)$ $b = \text{Vote}(id, v)$ $\mathcal{L} \leftarrow^U \{\text{TraceBallot}(b)\}$ return b
$\mathcal{O}\text{cast}(id, b)$ if $\text{Valid}(b) = 0$ or $ b < n - t_{\text{corr}}$ then return $\perp$ else $\text{Append}(BB_0, b); \text{Append}(BB_1, b)$	$\mathcal{O}\text{append}(b_0, b_1)$ if $\nexists T \in \mathcal{L} : \text{TraceBallot}(b_i) \subset T, \text{ for } i = 0, 1$ or $\text{Valid}(b_0) = 0$ or $\text{Valid}(b_1) = 0$ or $ b_0 < n - t_{\text{corr}}$ or $ b_1 < n - t_{\text{corr}}$ then return $\perp$ $\text{Append}(BB_0, b_0); \text{Append}(BB_1, b_1)$
$\mathcal{O}\text{tally}()$ $(r_0, \Pi_0) \leftarrow \text{Tally}(BB_0, sk)$ $(r_1, \Pi_1) \leftarrow \text{Tally}(BB_1, sk)$ return (r_0, r_1)	$\mathcal{O}\text{board}()$ return $\text{Publish}(BB_0); \text{Publish}(BB_1)$

Fig. 4: Threshold correctness experiment $\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{corr}, t_{\text{corr}}}(\lambda)$ which outputs 1 only if $r_0 \neq r_1$ [7].

C Security of the Voting Schemes

In this section, we prove that our voting schemes described in the previous section are threshold correct (Section C.1) and threshold receipt-free (Section C.2).

C.1 Threshold Correctness

Threshold correctness is captured by the experiment $\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{corr}, t_{\text{corr}}}(\lambda)$ (Figure 4), which models the integrity of election outcomes in the presence of adversarially modified ballots. The experiment maintains two internally consistent election views, each initialized with a set of honestly generated ballots and corresponding tracing information. The adversary $\mathcal{A}$ may attempt to manipulate these ballots by reconstructing new valid ones (via $\mathcal{O}\text{append}$) that omit up to t_{corr} shares and submitting them into both views. The adversary can also introduce arbitrary valid ballots of its own (via $\mathcal{O}\text{cast}$) and query the public bulletin board and tally results for each view. The experiment outputs 1 if the two election views yield different final results—indicating a breach of threshold correctness. A secure system ensures that such an event occurs only with negligible probability.

Theorem C.1 (Threshold Correctness). *Let TREnc be traceable and verifiable, and assume that the underlying NIZK proof systems are correct. Then both constructions achieve threshold correctness with $t_{\text{corr}} = n - t$ under a t -out-of- n secret sharing scheme. More precisely, for any efficient adversary $\mathcal{A}$ making q ballot-append queries, $\Pr[\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{corr}, t_{\text{corr}}}(\lambda) = 1] \leq qn\varepsilon_{\text{trace}} + \varepsilon_{\text{corr}}$ for MiniMix and $\Pr[\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{corr}, t_{\text{corr}}}(\lambda) = 1] \leq 2qln\varepsilon_{\text{trace}} + \varepsilon_{\text{corr}}$ for HomoRand, where l is the number of bits needed to represent the span of the vote domain (defined in Section 3.2), and $\varepsilon_{\text{trace}}, \varepsilon_{\text{corr}}$ bound $\mathcal{A}$'s advantage against the traceability of TREnc and the correctness error of the underlying MPC protocol, respectively.*

Proof. In the experiment $\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{corr}, t_{\text{corr}}}(\lambda)$, to append ballots to BB_0 and BB_1 , $\mathcal{A}$ can make $\mathcal{O}\text{cast}$ and $\mathcal{O}\text{append}$ queries.

$\mathcal{O}\text{cast}$: This oracle appends an identical valid ballot to both bulletin boards, as in [10]. Since the same ballot appears on both, this does not aid the adversary in winning the game, as it has no effect on the tally outcomes. This invariance follows from the fact that recombination is performed deterministically over all valid ballot shares.

$\mathcal{O}\text{append}$: Upon this query, two distinct valid ballots, $\mathbf{b}_0$ and $\mathbf{b}_1$, derived in a malicious yet valid manner from an honestly generated ballot $\mathbf{b}$, are appended to BB_0 and BB_1 , respectively. These ballots are required to be traceable to $\mathbf{b}$ while omitting at most t_{corr} shares. We analyze both constructions:

- **MiniMix construction:** Each vote share is encrypted under TREnc and labeled with a trace value. Given traceability, any valid share reusing the same trace must encrypt the same plaintext. Thus, the adversary cannot produce two valid ballots with the same trace that decrypt to different vote shares, except with probability at most $\varepsilon_{\text{trace}}$ per share. Since each ballot has n shares and at most q such append operations are allowed, the total advantage is bounded by $qn\varepsilon_{\text{trace}}$.
- **HomoRand construction:** Each vote share $\mathbf{b}_{ji}$ (for $j \in [l], i \in [n]$) includes two TREnc ciphertexts along with a proof of consistency (as detailed in Figure 3 and Section A). By traceability, the adversary cannot alter the encrypted messages in the two ciphertext. The adversary can at most break traceability on either ciphertext, giving a per-share failure bound of $2\varepsilon_{\text{trace}}$. As for the consistency proof, while the adversary may attempt arbitrary modifications, these do not influence the underlying encrypted values in the two ciphertexts, and thus do not affect the soundness of the game. Since there are ln such shares per ballot and q queries, the total advantage is bounded by $2qln\varepsilon_{\text{trace}}$.

In both constructions, the aggregation of vote shares via the **Combine** function operates on inputs provided by **PreTally**, which are guaranteed to be honest due to the traceability of TREnc and the correctness of the underlying secret sharing scheme. Specifically, any subset of at least $n - t_{\text{corr}}$ shares suffices to reconstruct the original vote. Moreover, by the correctness of the MPC-MiniMix and MPC-HomoRand procedures (established in Section B), the **PreTally** function deterministically outputs 1 if the reconstructed vote lies within the valid range, and 0 otherwise. Since both views include valid reconstructions and run the same tallying logic, the outputs are necessarily identical unless the adversary has introduced inconsistency via traceability failure.

As a consequence, the adversary’s advantage in distinguishing the games is bounded by the traceability error. Specifically, we obtain $\Pr[\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{corr}, t_{\text{corr}}}(\lambda) = 1] \leq qn\varepsilon_{\text{trace}} + \varepsilon_{\text{corr}}$ for the MiniMix construction and $\Pr[\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{corr}, t_{\text{corr}}}(\lambda) = 1] \leq 2qln\varepsilon_{\text{trace}} + \varepsilon_{\text{corr}}$ for the HomaRand construction by a standard guessing technique.

C.2 Threshold Receipt-Freeness

Threshold RF is defined via two experiments: $\text{Exp}_{\mathcal{A}, \mathcal{V}, \mathbf{t}_{\text{rf}}}^{\text{deceive}}(\lambda)$ and $\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{rf}, \mathbf{t}_{\text{rf}}, \beta}(\lambda)$ [10].

Definition C.1 (Threshold receipt-freeness). *A voting system $\mathcal{V}$ with n ballot processing servers has receipt-freeness with threshold $\mathbf{t}_{\text{rf}} \leq n$ if*

1. *There exists an algorithm Deceive such that, for every PPT adversary $\mathcal{A}$, $\Pr[\text{Exp}_{\mathcal{A}, \mathcal{V}, \mathbf{t}_{\text{rf}}}^{\text{deceive}}(\lambda) = 1]$ negligible in λ . (The experiment is defined in Figure 5.)*
2. *There exist algorithms SimSetupElection and SimProof such that, for every PPT adversary $\mathcal{A}$, the following advantage is negligible in λ*

$$\text{Adv}_{\mathcal{A}, \mathcal{V}}^{\text{rf}, \mathbf{t}_{\text{rf}}}(\lambda) = \left| \Pr \left[\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{rf}, \mathbf{t}_{\text{rf}}, 0}(\lambda) = 1 \right] - \Pr \left[\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{rf}, \mathbf{t}_{\text{rf}}, 1}(\lambda) = 1 \right] \right|,$$

where the experiment $\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{rf}, \mathbf{t}_{\text{rf}}, \beta}(\lambda)$ is defined in Figure 6.

The former formalizes the notion that even under coercion or external pressure to vote for a specific candidate v_0 , a voter can still successfully cast their intended vote v_1 without detection, despite partial system compromise. The adversary $\mathcal{A}$, given the public key and control over up to $\mathbf{t}_{\text{rf}}$ ballot processing servers, selects two votes v_0 and v_1 , as well as a random coins, and attempts to construct a convincing receipt that proves the ballot encodes v_0 . However, $\mathcal{A}$ observes only the ballot shares corresponding to the corrupted servers and the public tracking information. The goal is to use the Deceive algorithm to compute the remaining ballot shares such that the resulting ballot is valid for v_1 but remains indistinguishable from a ballot for v_0 with respect to the adversary's view.

```

 $\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{deceive}, \mathbf{t}_{\text{rf}}}(\lambda)$ 
(pk, sk)  $\leftarrow$  SetupElection( $1^\lambda$ )
( $v_0, v_1, \rho, I$ )  $\leftarrow$   $\mathcal{A}(\text{pk})$ 
if  $I \not\subseteq [n]$  or  $|I| > \mathbf{t}_{\text{rf}}$  then return 0
 $\mathbf{b}_0 \leftarrow \text{Vote}(\text{id}, v_0, \rho)$ 
 $\mathbf{b}_1 \leftarrow \text{Deceive}(\text{id}, v_0, v_1, \rho, I)$ 
if  $\{\mathbf{b}_0^i\}_{i \in I} \neq \{\mathbf{b}_1^i\}_{i \in I}$  or  $\mathbf{b}_1 \notin \text{Vote}(\text{id}, v_1)$ 
  or  $\text{TraceBallot}(\mathbf{b}_0) \neq \text{TraceBallot}(\mathbf{b}_1)$ 
  then return 1
return 0

```

Fig. 5: Deceive experiment.

The $\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{rf}, \mathbf{t}_{\text{rf}}, \beta}(\lambda)$ captures the intuition that even if a voter controls up to $\mathbf{t}_{\text{rf}}$ ballot processing servers, they should be unable to construct a ballot—possibly sampled from an arbitrary distribution—that can serve as a convincing receipt.

$\mathcal{O}\text{init}(\lambda)$	$\mathcal{O}\text{receiptLR}(\text{id}, B_0, B_1, B_2)$
if $\beta = 0$ then $(pk, sk) \leftarrow \text{SetupElection}(1^\lambda)$ else $(pk, sk, \tau) \leftarrow \text{SimSetupElection}(1^\lambda)$ $BB_0 \leftarrow \perp; BB_1 \leftarrow \perp$ return pk	if $ B_0 \neq B_1 $ or $ B_1 + B_2 > n$ or $ B_2 > t_{\text{rf}}$ then return $\perp$ if $\text{TraceBallot}(B_0 B_2) \neq \text{TraceBallot}(B_1 B_2)$ or $\text{Valid}(B_0 B_2) = 0$ or $\text{Valid}(B_1 B_2) = 0$ then return $\perp$ else $\text{Append}(BB_0, \text{ProcessBallot}(B_0) B_2)$ and $\text{Append}(BB_1, \text{ProcessBallot}(B_1) B_2)$
$\mathcal{O}\text{tally}()$	$\mathcal{O}\text{board}()$
$(r, \Pi) \leftarrow \text{Tally}(BB_0, sk)$ if $\beta = 1$ then $\Pi \leftarrow \text{SimProof}(BB_1, r)$ return (r, Π)	return $\text{Publish}(BB_\beta)$

Fig. 6: Threshold receipt freeness oracles from experiment $\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{rf}, t_{\text{rf}}, \beta}(\lambda)$, for $\beta = 0, 1$.

The experiment runs with a hidden bit $\beta \in \{0, 1\}$. When $\beta = 0$, the system operates honestly; when $\beta = 1$, certain components, such as the tally and its associated proof, are simulated using trapdoor information. The adversary is allowed to submit two valid ballots that differ only in the ballot shares processed by honest randomizers, along with known ballot shares corresponding to the compromised t_{rf} servers, which is describe by the $\mathcal{O}\text{receiptLR}$ oracle:

- $\mathcal{O}\text{receiptLR}$ allows the adversary to cast valid ballots and query the honest ballot processing servers to process their respective ballot pieces. Specifically, for a given voter id and input $(\text{id}, B_0, B_1, B_2)$, the oracle runs the ProcessBallot procedure on the valid ballot pieces B_0 and B_1 if they share the same traces for the same index. The procedure outputs B'_0 and B'_1 . As long as $|B_0 \cup B_2| = |B_1 \cup B_2| \leq n$ and $|B_2| \leq t_{\text{rf}}$, $b_0 = B'_0||B_2$ is appended to BB_0 and $b_1 = B'_1||B_2$ is appended to BB_1 . Up to reordering, we can always assume that the first servers are honest. B_2 represents the ballot pieces for which the malicious vote seller and the corrupt servers together know their whole content.

These inputs are used to populate two internal ballot boxes, BB_0 and BB_1 , which are updated in parallel. However, the adversary only observes the public bulletin board corresponding to BB_β . At any point, it may inspect this view, and eventually it queries the tally oracle to obtain the election outcome and a proof of its correctness, which is simulated if $\beta = 1$. The adversary's task is to guess the bit β . Security holds if no efficient adversary can distinguish between the real and simulated executions with advantage significantly better than random guessing.

Receipt-Freeness of our Voting Constructions. As outlined in Section 5.1, the tallying procedure in our protocols comprises two phases: (i) a PreTally phase that discards encrypted votes falling outside the valid domain, and (ii) the computation of the final election outcome, along with a publicly verifiable proof of correctness via a standard tallying mechanism, e.g., homomorphic aggregation or a verifiable mixnet. Since the latter follows well-understood simulation strate-

gies, our proof focuses on simulating the **PreTally** phase, which is the component that distinguishes our constructions from prior work.

The MiniMix Construction

To make the proof concrete, we instantiate the encryption scheme using the **TREnc** mechanism from [7], also described in Section A.3. More precisely, let $\text{PE} = (G, f, g, h) \in \mathbb{G}^4$ be the public parameters, with $f = g^\alpha h^\beta$ for a secret key $\text{SK} = (\alpha, \beta)$. A message m encrypted with randomness r yields the ciphertext $\mathbf{c} = (c_0, c_1, c_2) = (G^m f^r, g^r, h^r)$. Decryption is performed as $G^m = c_0 \cdot c_1^{-\alpha} \cdot c_2^{-\beta}$. In the threshold setting, each tallier $\mathcal{T}_i$, for $i \in [T]$, holds a share $\text{SK}_i = (\alpha_i, \beta_i)$ from a k -out-of- T secret sharing scheme (see Section 2). To decrypt a ciphertext, each tallier provides a partial decryption share D_i and a zero-knowledge proof of correctness.

Our receipt-freeness proof is structured in three steps. First, Theorem C.2 shows that the threshold decryption procedure is simulation-sound, assuming the underlying proof system is zero-knowledge. That is, the full decryption transcript, including partial decryptions and their proofs, can be simulated without knowledge of the secret key or plaintext. This result applies to both the **PreTally** and final **Tally** phases. Building on this, Theorem C.3 establishes that the **PreTally** procedure is also secure. Finally, Theorem C.4 combines both results to conclude the threshold receipt-freeness of the entire construction.

Theorem C.2. *Let the threshold decryption procedure, instantiated as above, be augmented with a zero-knowledge proof system that demonstrates its correctness. Suppose the advantage of any adversary in distinguishing between real and simulated proofs in this system is at most ε_{ZK} , then the decryption leaks no auxiliary information beyond the decryption result itself, except with probability at most ε_{ZK} .*

Proof. We construct a simulator $\mathcal{S}_1$ that simulates the view of an adversary $\mathcal{A}$ during the threshold decryption phase. The simulation is performed using only public parameters, the ciphertext $\mathbf{c}$, and the final decryption result R , without access to the honest talliers' secret keys. We show that the resulting simulated transcript is computationally indistinguishable from a real execution, assuming the zero-knowledge property of the underlying proof system.

Let the adversary $\mathcal{A}$ corrupt up to $k-1$ talliers, denoted by the set $\mathcal{C} \subseteq \{\mathcal{T}_1, \dots, \mathcal{T}_T\}$, where $k < T/2$ is the threshold. Let $\mathcal{H} = \mathcal{T} \setminus \mathcal{C}$ denote the set of honest talliers. Without loss of generality, we assume that exactly $k-1$ talliers are corrupted, i.e., $|\mathcal{C}| = k-1$; otherwise, the simulator can synthetically generate shares for additional corrupted parties to match this bound.

Inputs to $\mathcal{S}_1$:

- The public elements PE , ciphertext $\mathbf{c} = (c_0, c_1, c_2)$, decrypted result R , and the secret shares $\text{SK}_i = (\alpha_i, \beta_i)$ of corrupted talliers $\mathcal{T}_i \in \mathcal{C}$.

- The trapdoor of ZK simulation for the employed proof system, depending on which ZK proof system used. For instance, in a Chaum-Pedersen protocol [5] instantiated with the Fiat-Shamir heuristic in the Random Oracle Model, the trapdoor is the ability to program the random oracle.

Output of $\mathcal{S}_1$: A simulated transcript that is computationally indistinguishable from the real execution. It consists of:

- Decryption shares D_j and zero-knowledge proofs π_j for all corrupted talliers $\mathcal{T}_j \in \mathcal{C}$ (generated honestly using known keys).
- Simulated decryption shares D_i and simulated proofs π'_i for all honest talliers $\mathcal{T}_i \in \mathcal{H}$.

Simulation procedure:

- For each corrupted tallier $\mathcal{T}_j \in \mathcal{C}$, $\mathcal{S}_1$ uses the known secret keys to compute $D_j = c_1^{\alpha_j} \cdot c_2^{\beta_j}$ and includes their real proofs, which the adversary is allowed to provide.
- For each honest tallier $\mathcal{T}_i \in \mathcal{H}$, $\mathcal{S}_1$ computes a decryption share D_i that is algebraically consistent with the known decryption result R , as follows:

$$D_i = \left(\frac{c_0}{G^R \cdot \prod_{\mathcal{T}_j \in \mathcal{C}} D_j^{\Lambda(j)}} \right)^{1/\Lambda(i)}$$

where $\Lambda(i)$ are Lagrange interpolation coefficients at 0 for index i , used to reconstruct the secret from the shares of all corrupted parties and the honest one $\mathcal{T}_i$.

- Then, $\mathcal{S}_1$ invokes the ZK proof simulator (using the trapdoor) to generate simulated proofs π'_i proving the correctness of D_i , without actually knowing the secret shares.

By construction, the simulated decryption shares of the honest talliers are consistent with the known decryption result R , and the proofs π'_i are indistinguishable from real proofs by the zero-knowledge property of the proof system. The adversary's view of the simulated execution is thus computationally indistinguishable from a real execution, except with probability at most ε_{ZK} . $\square$

Theorem C.3. *Let TREnc be a publicly verifiable threshold encryption scheme, and assume that the proof system used in the threshold decryption phase is zero-knowledge with distinguishing advantage at most ε_{ZK} . Then, the PreTally phase (Algorithm 1) is simulation-based secure: it leaks no information beyond the output bit $b \in \{0, 1\}$, except with probability at most ε_{ZK} .*

Proof. We show that the PreTally phase admits a simulator whose output is computationally indistinguishable from a real execution, assuming the zero-knowledge property of the underlying proof system. Informally, the goal is to

ensure that no information about the encrypted vote is leaked beyond whether or not it is in the valid range.

We adopt the same corruption model as in Theorem C.2: the adversary $\mathcal{A}$ may statically corrupt up to $k-1$ talliers, denoted by $\mathcal{C} \subseteq \{1, \dots, T\}$, where $|\mathcal{C}| \leq k-1$.

The **PreTally** procedure comprises the aggregation of ciphertexts via homomorphic operations followed by an execution of the MPC-MiniMix protocol, which consists of a verifiable shuffle (Algorithm 1, lines 3–18) and a threshold decryption (lines 19–23). Since the aggregation is deterministic and publicly verifiable, we focus on simulating the MPC-MiniMix protocol.

Inputs to $\mathcal{S}_2$:

- The ciphertext vector $\mathbf{C}^* = \{\mathbf{C}_j^*\}_{j=1}^d$, as specified in Figure 3.
- The output bit $b \in \{0, 1\}$ of the **PreTally** procedure, where $b = 1$ denotes a valid vote and $b = 0$ otherwise.
- The secret keys (α_i, β_i) for each corrupted tallier $\mathcal{T}_i \in \mathcal{C}$.
- The trapdoor of ZK simulation for the employed proof system.

Output of $\mathcal{S}_2$: A transcript that is computationally indistinguishable from a real execution, including:

- The adversary’s input and output messages during the shuffle phase.
- Simulated decryption shares and zero-knowledge proofs for honest talliers during the threshold decryption phase, consistent with the output bit b .

Simulation procedure:

1. Shuffle.
 - For each corrupted tallier $\mathcal{T}_i \in \mathcal{C}$, the adversary provides $(\mathbf{C}^{(k)}, \pi^{(k)})$. If a proof fails verification, the party is excluded, as specified in Section 3.1.
 - For honest talliers, the simulator faithfully reproduces their behavior (e.g., proper shuffle and valid proofs), since it does not affect the adversary’s view beyond what is already public.

2. Threshold Decryption

The simulator $\mathcal{S}_2$ invokes subroutine $\mathcal{S}_1$ to simulate the threshold decryption phase without knowledge of the honest talliers’ secret keys but with access to the final output bit b .

- If $b = 1$ (Valid vote): **PreTally** is expected to output 1 if at least one ciphertext $C_j^{(T)}$ decrypts to 0. The simulator samples a random index $i^* \in [d]$ and invokes $\mathcal{S}_1$ from the Theorem C.2 to simulate decryption shares and ZKPoKs such that $\text{TREnc.Dec}(\text{SK}, C_{i^*}^{(T)}) = 0$ and $\text{TREnc.Dec}(\text{SK}, C_i^{(T)}) = r_i$ for $i < i^*$ where r_i are also chosen by the simulator, i.e., $r_i \leftarrow \mathbb{Z}_p \setminus \{0\}$. Since the **PreTally** procedure halts as soon as a zero is decrypted, decryption of ciphertexts with index $i > i^*$ is omitted. The output distribution of the simulated index i^* and the values r_i is computationally indistinguishable from a real execution, by the zero-knowledge property of the proof system.

- If $b = 0$ (Invalid vote): $\mathcal{S}_2$ samples $r_i \leftarrow \mathbb{Z}_p \setminus \{0\}$ for all $i \in [d]$ and calls the simulator $\mathcal{S}_1$ to simulate decryption shares and proofs such that $\text{TREnc.Dec}(\text{SK}, C_i^{(T)}) = r_i$. Since no decryption results in 0, the output of **PreTally** is correctly simulated.

In both cases, the simulation is computationally indistinguishable from a real execution under the zero-knowledge assumption. Hence, the adversary's distinguishing advantage is bounded by ε_{ZK} . $\square$

Theorem C.4. *Let TREnc be a TCCA-secure and verifiable threshold encryption scheme, and suppose the PreTally phase is secure. Then the MiniMix construction achieves threshold receipt-freeness under a t -out-of- n sharing scheme with threshold $t_{\text{rf}} = t-1$. More precisely, for any PPT adversary $\mathcal{A}$ issuing at most q ballot-append queries to the $\mathcal{O}\text{receiptLR}$ oracle, we have:*

$$\Pr[\text{Exp}_{\mathcal{A}, \mathcal{V}, t_{\text{rf}}}^{\text{deceive}}(\lambda) = 1] \leq \varepsilon_{\text{verif}}, \quad \text{Adv}_{\mathcal{A}, \mathcal{V}}^{\text{rf}, t_{\text{rf}}, \beta}(1^\lambda) \leq \varepsilon_{\text{ZK}} + q(n - t_{\text{rf}})\varepsilon_{\text{tcca}},$$

where ε_{ZK} , $\varepsilon_{\text{verif}}$, and $\varepsilon_{\text{tcca}}$ bound the adversarial advantage against the proof systems in the pre-tally phase, verifiability, and TCCA-security of TREnc, respectively.

Proof. **The experiment** $\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{deceive}, t_{\text{rf}}}(\lambda)$. Security follows directly from traceability and verifiability of TREnc, and the correctness of the secret sharing scheme as shown in [10].

The experiment $\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{rf}, t_{\text{rf}}, \beta}(\lambda)$. We now analyze $\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{rf}, t_{\text{rf}}, \beta}(\lambda)$ via a sequence of hybrid games, transitioning from $\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{rf}, t_{\text{rf}}, 0}(\lambda)$ to $\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{rf}, t_{\text{rf}}, 1}(\lambda)$. The adversary $\mathcal{A}$ outputs two valid ballots $\text{CT}_0 = \text{B}_0 \parallel \text{B}_2$ and $\text{CT}_1 = \text{B}_1 \parallel \text{B}_2$, encoding distinct votes v_0 and v_1 , such that both ballots pass verification and have equal traces:

$$\text{Valid}(\text{CT}_0) = \text{Valid}(\text{CT}_1) = 1, \quad \text{TraceBallot}(\text{CT}_0) = \text{TraceBallot}(\text{CT}_1).$$

Let X_i denote the adversary's success probability in Game i .

Game₀(λ): We define **Game₀**(λ) as the original receipt-freeness experiment. By definition, $\Pr[X_0] = \text{Adv}(\mathcal{A})$.

Game₁(λ): This game is identical to **Game₀**, except that we simulate the election setup using $\mathcal{S}_1$ from Theorem C.2, which generates simulated zero-knowledge proofs for honest talliers. By the ZK property, this change is indistinguishable except with probability at most ε_{ZK} , hence $|\Pr[X_1] - \Pr[X_0]| \leq \varepsilon_{\text{ZK}}$.

Game₂(λ): This game proceeds as **Game₁**(λ), except that the decryption is performed using $\text{Dec}(\text{SK}, \cdot)$ of TREnc in each call to $\mathcal{O}\text{receiptLR}$ before the randomization in **ProcessBallot**. The election outcome is then computed from the resulting function. Since the view in Game 2 is identical to that of Game 1, we have $\Pr[X_2] = \Pr[X_1]$.

Game₃(λ): This game modifies the response behavior to adversarial queries to the $\mathcal{O}\text{receiptLR}$ oracle. We define a hybrid sequence **Game_{3,i}** where $i \in \{1, \dots, n - t_{\text{rf}}\}$ denotes the number of processed components of B_0 replaced by those from B_1 within BB_0 . More precisely,

Game_{3,1}(λ): Instead of re-randomizing the first ciphertext in B_0 , we re-randomize $CT_{1,1}$: i.e., $B'_0 = (CT'_{1,1}, CT'_{0,2}, \dots, CT'_{0,n-t_{\text{rf}}})$. Since all ciphertexts are valid under TREnc , the adversary can distinguish this change only if it can tell whether $CT'_{1,1}$ or $CT'_{0,1}$ was randomized. This advantage is bounded by the TREnc-TCCA security, so $|\Pr[X_{3,1}] - \Pr[X_2]| \leq \varepsilon_{\text{tcca}}$.

The remainder of the protocol follows the **PreTally** procedure from Figure 2, except that, the MPC-MiniMix algorithm is now simulated using $\mathcal{S}_2$ from Theorem C.3, where it receives:

- The ciphertext vector $C^* = \{C_j^*\}_{j=1}^d$, where $C_j^* = C_0^* \cdot \text{CPA}(\text{PK}, -v_j)$, and $C_0^* = \text{Combine}(n, t, CT'_0)$ for the re-randomized ballot $CT'_0 = B'_0 \parallel B_2$.
- The expected output bit $b = \text{PreTally}(\cdot, \cdot, \bar{CT}_0)$.

The simulator $\mathcal{S}_2$ must produce the validity outcome of the original ballot $\bar{CT}_0 = (CT'_{0,1}, CT'_{0,2}, \dots, CT'_{0,n-t_{\text{rf}}}) \parallel B_2$, known by TREnc.Dec (as introduced in the previous game), along with simulated proofs for the honest talliers' actions in the pre-tally phase. By Theorem C.3, the outputs produced by $\mathcal{S}_2$ are computationally indistinguishable from those in a real protocol execution, owing to the zero-knowledge property of the proof system. Thus, we have $|\Pr[X_{3,1}] - \Pr[X_2]| \leq \varepsilon_{\text{tcca}}$.

Game_{3,i}(λ): In this game, the first i ciphertexts in B'_0 are replaced with those from B'_1 . Each step transitions from **Game_{3,i-1}** $\rightarrow$ **Game_{3,i}**, by replacing one ciphertext $CT'_{0,i}$ with $CT'_{1,i}$. By the TCCA-security of TREnc , the adversary's distinguishing advantage in each step is bounded:

$$|\Pr[X_{3,i}] - \Pr[X_{3,i-1}]| \leq \varepsilon_{\text{tcca}}.$$

After $(n - t_{\text{rf}})$ replacements, we obtain a final ballot B'_0 identical to B'_1 , i.e., $CT'_0 = CT'_1$ or i.e., the view from $\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{rf}, t_{\text{rf}}, 1}(\lambda)$.

Summing the differences across the hybrid steps, we obtain:

$$|\Pr[X_0] - \Pr[X_3]| \leq \varepsilon_{\text{ZK}} + (n - t_{\text{rf}})\varepsilon_{\text{tcca}}.$$

Extending this to q queries to $\mathcal{O}\text{receiptLR}$ via a standard hybrid argument yields the final bound: $\text{Adv}_{\mathcal{A}, \mathcal{V}}^{\text{rf}, t_{\text{rf}}, \beta}(1^\lambda) \leq \varepsilon_{\text{ZK}} + q(n - t_{\text{rf}})\varepsilon_{\text{tcca}}$. $\square$

The HomoRand Construction

Theorem C.5. *Let TREnc be a TCCA-secure and verifiable threshold encryption scheme, and suppose the **PreTally** phase is secure. Then the **HomoRand** construction achieves threshold receipt-freeness under a t -out-of- n sharing scheme*

with threshold $t_{\text{rf}} = t-1$. More precisely, for any PPT adversary $\mathcal{A}$ issuing at most q ballot-append queries to the $\mathcal{O}\text{receiptLR}$ oracle, we have:

$\Pr[\text{Exp}_{\mathcal{A}, \mathcal{V}, t_{\text{rf}}}^{\text{deceive}}(\lambda) = 1] \leq \varepsilon_{\text{verif}}$, $\text{Adv}_{\mathcal{A}, \mathcal{V}}^{\text{rf}, t_{\text{rf}}, \beta}(1^\lambda) \leq \varepsilon_{\text{ZK}} + lq(n-t_{\text{rf}})(2\varepsilon_{\text{tcca}} + \varepsilon_{\text{sxdh}})$, where l is the bit-length of the vote domain, and ε_{ZK} , $\varepsilon_{\text{sxdh}}$, $\varepsilon_{\text{verif}}$, $\varepsilon_{\text{tcca}}$ bound the adversarial advantage against the ZK proof systems, the SXDH (Symmetric eXternal Diffie-Hellman) assumption, verifiability, and TCCA-security of TREnc, respectively.

Proof. The proof proceeds similarly to the one for the MiniMix construction. The receipt-freeness property is shown via a sequence of hybrid experiments, in which the response behavior to adversarial queries to the $\mathcal{O}\text{receiptLR}$ oracle by progressively replacing processed components of B_0 with those from B_1 within BB_0 .

The main difference lies in the handling of ballot shares: each share in the HomoRand construction includes an additional consistency proof, whose simulation under the SXDH assumption introduces an extra distinguishing advantage $\varepsilon_{\text{sxdh}}$ per share transition. Since each vote consists of l bits, and at most $q(n-t_{\text{rf}})$ such share transitions occur across the experiment, the total accumulated distinguishing probability is at most $lq(n-t_{\text{rf}}) \cdot \varepsilon_{\text{sxdh}}$.

Combining this with the security bounds inherited from TCCA-security and the zero-knowledge properties of the proof systems yields the claimed bound. $\square$

Correction and Clarification (Intuitive) on Verifiability Issue in HomoRand. As presented in Section 3.2, the core idea of HomoRand for checking the validity of an encrypted vote is to compute a combined masked sum:

$$\sum_{j=1}^l \gamma_j b_j (1 - b_j) \stackrel{?}{=} 0,$$

where γ_j are random masking values. In this way, if the vote is valid, the sum evaluates to 0; otherwise, the sum produces a random non-zero value. The γ_j masks ensure privacy, so no information about individual encrypted bits is leaked during decryption.

However, if all talliers (and possibly the voter) are corrupted, they could choose γ_j such that the sum equals 0 even for an invalid vote, or vice versa. To prevent this, we introduce an additional unpredictable mask γ'_j , which is defined after each tallier chose its mask value and a vote is chosen, i.e.,:

- Each tallier T_k commits to their chosen $\gamma_{j,k}$ in the pre-tally phase (commitments $d_{j,k}$).
- We define $\gamma'_j := \text{hash}(\{d_{j,k}\}_{k \in T} \parallel C')$, where C' is the final ciphertext of the voter after combination.

This γ'_j can then be publicly added to the Homorand check such that we will be able to verify:

$$\sum_j (\gamma_j + \gamma'_j) b_j (1 - b_j) \stackrel{?}{=} 0,$$

Thus, even if the voter, randomizers, and talliers collude (i.e., know all γ_j and the vote bits b_j):

- They cannot adjust the input to the hash to achieve a specific γ'_j , because the hash is a one-way function.
- The sum $\sum_{j=1}^l \gamma'_j b_j (1 - b_j)$ is a polynomial with coefficients $b_j(1 - b_j)$. By the Schwartz-Zippel lemma, the probability of the sum being 0 for an invalid vote is negligible.

Thus, the addition of γ'_j resolves the soundness issue while preserving privacy and verifiability, ensuring that the Homorand vote validation remains correct and secure against colluding parties. We will provide the complete proof with technical details in an updated version of this paper.